Министерство науки и высшего образования Российской Федерации
ФГБОУ ВО «Кубанский государственный технологический университет»
(ФГБОУ ВО «КубГТУ»)

Институт [наименование института]

Кафедра информационных систем и программирования

Направление подготовки 09.03.04 Программная инженерия

Профиль [наименование профиля]

КУРСОВАЯ РАБОТА

по дисциплине «Системы искусственного интеллекта»

на тему: «Разработка гибридной интеллектуальной системы планирования маршрутов доставки на основе генетического алгоритма и нечёткой логики»

Выполнил(-а) студент(-ка) [курс] курса [группа] группы
[Фамилия Имя Отчество]

Допущен к защите _____

Руководитель (нормоконтролёр) работы
[должность] _____ [Фамилия И.О.]

Защищен _____ Оценка _____

Члены комиссии _____

_____

_____

(подпись, дата, расшифровка подписи)

Краснодар
2026 г.

ФГБОУ ВО «Кубанский государственный технологический университет»
(ФГБОУ ВО «КубГТУ»)

Институт [наименование института]
Кафедра информационных систем и программирования
Направление подготовки 09.03.04 Программная инженерия
Профиль [наименование профиля]

УТВЕРЖДАЮ
Зав. кафедрой _____
«___» _____ 20__ г.

ЗАДАНИЕ
на курсовую работу

Студенту(-ке) [Фамилия И.О.] [курс] курса группы [группа]
Тема работы: «Разработка гибридной интеллектуальной системы планирования маршрутов доставки на основе генетического алгоритма и нечёткой логики»
(утверждена указанием директора института № ___ от _____ 20__ г.)

План работы:
1. Анализ проблем предметной области: задача маршрутизации транспорта с ограничениями вместимости и временными окнами, критерии качества плана доставки.
2. Выбор и обоснование интеллектуальных методов: генетический алгоритм и нечёткий логический вывод Мамдани; гибридная схема их взаимодействия.
3. Разработка интеллектуальной системы (программная реализация на Python) и экспериментальный анализ её работы.

Объём работы:
а) пояснительная записка 39 с.;
б) иллюстративная часть _____ лист(-ов).

Рекомендуемая литература:
1. Джонс, М. Т. Программирование искусственного интеллекта в приложениях / М. Т. Джонс ; пер. с англ. А. И. Осипова. – М. : ДМК Пресс, 2011. – 312 с.
2. Рутковская, Д. Нейронные сети, генетические алгоритмы и нечеткие системы / Д. Рутковская, М. Пилиньский, Л. Рутковский ; пер. с польск. И. Д. Рудинского. – М. : Горячая линия – Телеком, 2006. – 452 с.
3. Гладков, Л. А. Генетические алгоритмы / Л. А. Гладков, В. В. Курейчик, В. М. Курейчик ; под ред. В. М. Курейчика. – 2-е изд., испр. и доп. – М. : Физматлит, 2006. – 320 с.

Срок выполнения работы: с «___» _____ по «___» _____ 20__ г.
Срок защиты: «___» _____ 20__ г.
Дата выдачи задания: «___» _____ 20__ г.
Дата сдачи работы на кафедру: «___» _____ 20__ г.

Руководитель работы _____ [Фамилия И.О.]
(должность, подпись)

Задание принял студент _____ [Фамилия И.О.]
(подпись)

РЕФЕРАТ

Курсовая работа: 61 с., 12 рис., 8 табл., 20 источников, 4 приложения.

ИНТЕЛЛЕКТУАЛЬНАЯ СИСТЕМА, ГЕНЕТИЧЕСКИЙ АЛГОРИТМ, НЕЧЁТКАЯ ЛОГИКА, АЛГОРИТМ МАМДАНИ, ЛИНГВИСТИЧЕСКАЯ ПЕРЕМЕННАЯ, БАЗА ПРАВИЛ, ГИБРИДНАЯ СИСТЕМА, ЗАДАЧА МАРШРУТИЗАЦИИ ТРАНСПОРТА, ВРЕМЕННЫЕ ОКНА, ЛОГИСТИКА

Объект исследования – процесс планирования маршрутов доставки грузов парком транспортных средств с ограниченной грузоподъёмностью при наличии временных окон и приоритетов (срочности) заказов. Предмет исследования – гибридные методы искусственного интеллекта для поиска и оценки планов доставки.

Цель работы – разработка интеллектуальной системы, в которой генетический алгоритм ищет распределение заказов по машинам и порядок их объезда, а система нечёткого вывода Мамдани с базой из 26 экспертных правил оценивает качество каждого плана и тем самым задаёт функцию приспособленности.

Методы: генетический алгоритм с перестановочным кодированием, упорядоченным скрещиванием, тремя операторами мутации и элитизмом; нечёткая логика (лингвистические переменные, продукционная база правил с весовыми коэффициентами, алгоритм Мамдани с дефаззификацией по центру тяжести); вычислительный эксперимент.

Результат: программная система на языке Python с графическим и консольным интерфейсом. На задачах с 50 клиентами система сокращает суммарное опоздание по сравнению с жадным алгоритмом «ближайший сосед» на 95 %, а на малых задачах находит точный оптимум, подтверждённый полным перебором.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....5
НОРМАТИВНЫЕ ССЫЛКИ.....7
ТЕРМИНЫ, ОПРЕДЕЛЕНИЯ И СОКРАЩЕНИЯ.....8
1 Анализ проблем предметной области.....9
1.1 Логистика «последней мили» и задача планирования доставки.....9
1.2 Задача маршрутизации транспорта.....9
1.3 Вычислительная сложность задачи.....10
1.4 Критерии качества плана доставки и их конфликт.....10
1.5 Лицо, принимающее решение, и необходимые ему зависимости.....12
1.6 Постановка задачи курсовой работы.....12
2 Выбор и применение интеллектуального метода.....13
2.1 Обзор подходов искусственного интеллекта.....13
2.2 Обоснование выбора методов и гибридной схемы.....14
2.3 Генетические алгоритмы.....15
2.4 Нечёткая логика и алгоритм вывода Мамдани.....16
3 Разработка интеллектуальной системы.....18
3.1 Архитектура системы.....18
3.2 Модель задачи и генератор исходных данных.....19
3.3 Нечёткая модель оценки качества плана.....19
3.3.1 Лингвистические переменные.....19
3.3.2 База нечётких правил.....22
3.3.3 Реализация механизма вывода.....23
3.4 Генетический алгоритм.....25
3.4.1 Кодирование решения.....25
3.4.2 Функция приспособленности.....25
3.4.3 Генетические операторы.....26
3.4.4 Начальная популяция, элитизм и останов.....27
4 Экспериментальный анализ.....28
4.1 Методика экспериментов.....28
4.2 Сравнение с базовыми методами.....28
4.3 Сравнение с полным перебором.....30
4.4 Сходимость генетического алгоритма.....31
4.5 Влияние параметров генетического алгоритма.....32
4.6 Пример работы системы и протокол нечёткого вывода.....34
4.7 Работа программы.....36
ЗАКЛЮЧЕНИЕ.....37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....38
Приложение А Листинг основных модулей программы.....40
Приложение Б Пример входного файла задачи.....58
Приложение В Пример протокола работы программы.....59
Приложение Г Диаграмма классов интеллектуальной системы.....61

ВВЕДЕНИЕ

Доставка товаров конечному получателю – «последняя миля» – самое дорогое звено логистической цепочки. Диспетчер службы доставки ежедневно распределяет десятки заказов между машинами и определяет порядок объезда, учитывая грузоподъёмность транспорта, временные окна клиентов, приоритет срочных заказов и равномерность загрузки водителей. Число вариантов плана исчисляется миллиардами, а критерии качества противоречат друг другу, поэтому план, составленный вручную или простыми правилами, ведёт к перерасходу пробега и опозданиям.

Моделью этой задачи служит задача маршрутизации транспортных средств (VRP) с ограничениями вместимости и временными окнами – NP-трудная задача [1], для которой точные методы применимы лишь при малой размерности. Поэтому используются метаэвристики, в частности генетические алгоритмы [2, 3]. Генетическому алгоритму нужна функция приспособленности, а качество плана многокритериально и во многом субъективно: логист оценивает план словами «пробег великоват, но опозданий нет». Формализовать такие оценки позволяет нечёткая логика [4, 5].

Актуальность работы определяется потребностью служб доставки в автоматическом построении планов, учитывающих противоречивые критерии, и в объяснимых оценках таких планов.

Объект исследования – процесс планирования маршрутов доставки грузов парком транспортных средств.

Предмет исследования – гибридные методы искусственного интеллекта (генетический алгоритм и нечёткий логический вывод) для поиска и оценки планов доставки.

Цель работы – разработать гибридную интеллектуальную систему, в которой генетический алгоритм ищет решение задачи маршрутизации (распределение заказов по машинам и порядок объезда), а нечёткий вывод по

алгоритму Мамдани вычисляет оценку качества маршрута, служащую функцией приспособленности генетического алгоритма.

Для достижения цели поставлены следующие **задачи**:

– проанализировать предметную область, сформулировать задачу маршрутизации с временными окнами, критерии качества плана и зависимости между ними;

– выполнить обзор методов искусственного интеллекта и обосновать выбор генетического алгоритма и нечёткой логики, а также гибридной схемы их взаимодействия;

– разработать нечёткую модель оценки качества плана: лингвистические переменные, функции принадлежности и базу продукционных правил с весовыми коэффициентами;

– разработать генетический алгоритм: способ кодирования решения, генетические операторы, критерии останова;

– реализовать программную систему с графическим и консольным интерфейсом;

– провести экспериментальный анализ: сравнить систему с базовыми методами и с точным решением, исследовать влияние параметров генетического алгоритма.

Методы исследования: теория нечётких множеств и нечёткий логический вывод, эволюционные вычисления, имитационное моделирование движения транспорта, вычислительный эксперимент с последующей статистической обработкой результатов.

Практическая значимость: система может служить диспетчеру инструментом поддержки принятия решений – она строит план за секунды и объясняет оценку через сработавшие правила базы знаний. Генетический алгоритм и механизм нечёткого вывода реализованы на Python без сторонних библиотек искусственного интеллекта.

НОРМАТИВНЫЕ ССЫЛКИ

В настоящей курсовой работе использованы ссылки на следующие нормативные документы:

– ГОСТ Р 7.0.5-2008 СИБИД. Библиографическая ссылка. Общие требования и правила составления;

– ГОСТ 2.105-2019 ЕСКД. Общие требования к текстовым документам;

– ГОСТ 7.9-95 СИБИД. Реферат и аннотация. Общие требования;

– ГОСТ 7.82-2001 СИБИД. Библиографическая запись. Библиографическое описание электронных ресурсов. Общие требования и правила составления;

– ГОСТ 19.701-90 ЕСПД. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения.

ТЕРМИНЫ, ОПРЕДЕЛЕНИЯ И СОКРАЩЕНИЯ

В настоящей пояснительной записке применяют следующие термины с соответствующими определениями и сокращения.

Аккумуляция	– Этап нечёткого вывода: объединение (операция max) активизированных нечётких множеств всех правил в одно множество выходной переменной
Активизация	– Этап нечёткого вывода: вычисление степени истинности заключения правила с учётом весового коэффициента и «срез» функции принадлежности терма заключения
ГА	– Генетический алгоритм – эвристический метод поиска, имитирующий естественный отбор: популяция решений улучшается операторами отбора, скрещивания и мутации
Дефаззификация	– Переход от нечёткого множества к чёткому числу; в работе – метод центра тяжести
Лингвистическая переменная	– Переменная, значениями которой являются слова (термы), каждое из которых задано нечётким множеством
ЛПР	– Лицо, принимающее решение (диспетчер, логист службы доставки)
Приспособленность	– Числовая оценка качества особи, по которой ведётся отбор в ГА
Терм	– Значение лингвистической переменной («короткая», «средняя», «длинная»)
Фаззификация	– Вычисление степеней принадлежности чётких входных значений термам лингвистических переменных
Функция принадлежности	– Функция $\mu(x) \in [0; 1]$, задающая степень принадлежности элемента x нечёткому множеству
CVRP	– Capacitated Vehicle Routing Problem – задача маршрутизации с ограничением вместимости
OX	– Упорядоченное скрещивание (Order Crossover) – оператор скрещивания для перестановок
VRP	– Vehicle Routing Problem – задача маршрутизации транспортных средств
VRPTW	– Vehicle Routing Problem with Time Windows – задача маршрутизации с временными окнами

1 Анализ проблем предметной области

1.1 Логистика «последней мили» и задача планирования доставки

«Последняя миля» – завершающий этап доставки от склада до получателя: много мелких отправок по городу, жёсткие требования ко времени и высокая доля затрат на топливо и труд водителей. Служба доставки располагает складом и парком однотипных машин, которые утром загружаются, развозят заказы и к концу смены возвращаются. Диспетчер ежедневно распределяет заказы (адрес, объём, временное окно, срочность) по машинам без перегруза и определяет порядок объезда – составляет *план доставки*, от которого зависят себестоимость и уровень сервиса. В модели используются обозначения: клиент $i = 1, \ldots, N$ имеет объём заказа q_i, временное окно $[e_i; l_i]$ и срочность $u_i \in \{1, 2, 3\}$ (обычный, повышенный, срочный заказ); парк состоит из K одинаковых машин вместимостью Q; смена длится H = 480 мин; склад обозначается вершиной 0.

1.2 Задача маршрутизации транспорта

Моделью планирования доставки является задача маршрутизации транспортных средств (VRP) Дж. Данцига и Дж. Рамсера [6]. Различают варианты с ограничением вместимости (CVRP), с временными окнами (VRPTW) [7] и многокритериальные постановки [8]. Пусть $x_{ijk} = 1$, если машина k едет из точки i непосредственно в точку j, а d_{ij} – расстояние между ними. Классическая CVRP требует минимизировать суммарный пробег

$$min \sum_{k=1}^{K} \sum_{i,} \sum_{j} d_{ij} \cdot x_{ijk} \tag{1}$$

при ограничении вместимости каждого маршрута R_k

$$\sum_{i \in R_k} q_i \leq Q, k = 1, \ldots, K \tag{2}$$

и условии, что каждый клиент посещается ровно один раз. При наличии временных окон момент начала обслуживания s_i и опоздание

определяются моментом прибытия t_i: при раннем прибытии машина ждёт открытия окна, при позднем возникает опоздание:

$$s_i = max(e_i, t_i), late_i = max(0, s_i - l_i) \quad (3)$$

В жёсткой постановке VRPTW опоздания запрещены, однако на практике окна обычно «мягкие»: опоздание допускается, но ухудшает сервис. В данной работе вместимость – жёсткое ограничение, временные окна – мягкие.

1.3 Вычислительная сложность задачи

Задача VRP обобщает задачу коммивояжёра и является NP-трудной [1]: не известно алгоритма, время работы которого росло бы полиномиально с ростом N. Если записать план как перестановку N клиентов, разделённую K − 1 одинаковыми разделителями на K маршрутов, число различных записей равно

$$S(N, K) = \frac{(N + K - 1)!}{(K - 1)!} \quad (4)$$

Уже при N = 20, K = 3 это около $5{,}6 \cdot 10^{20}$ вариантов: перебор со скоростью 10^9 вариантов в секунду занял бы около 18 тысяч лет, а при N = 50, K = 7 (около 10^{72} вариантов) – порядка $3 \cdot 10^{55}$ лет. Точные методы применимы лишь к задачам с несколькими десятками клиентов и требуют линейной целевой функции [8], поэтому используются быстрые однокритериальные эвристики («ближайший сосед») и метаэвристики (генетические алгоритмы, имитация отжига), допускающие произвольную целевую функцию [9].

1.4 Критерии качества плана доставки и их конфликт

Качество плана не сводится к пробегу. Анализ требований служб доставки позволяет выделить пять критериев (таблица 1).

Таблица 1 – Критерии качества плана доставки

Критерий	**Показатель в модели**	**Желаемое значение**	**Кому важен**
Протяжённость маршрутов	Суммарный пробег D, отнесённый к пробегу эталонного решения	Меньше	Бухгалтерия (топливо, износ)
Соблюдение временных окон	Средняя относительная величина опоздания	Нулевая	Клиенты, отдел продаж
Обслуживание срочных заказов	Среднее время доставки срочных заказов в долях смены	Раньше	Клиенты со срочными заказами
Загрузка транспорта	Максимальная загрузка машины в долях вместимости	Не более 1	Водители, безопасность
Сбалансированность	Коэффициент вариации длительностей маршрутов	Близок к 0	Водители, отдел кадров

Критерии конфликтуют (рисунок 1). Кратчайший объезд соседних клиентов игнорирует их окна; доставка срочных заказов первыми задерживает обычных клиентов; выравнивание маршрутов между водителями удлиняет их. Ограничение вместимости компромисса не допускает: перегруженный план неосуществим.

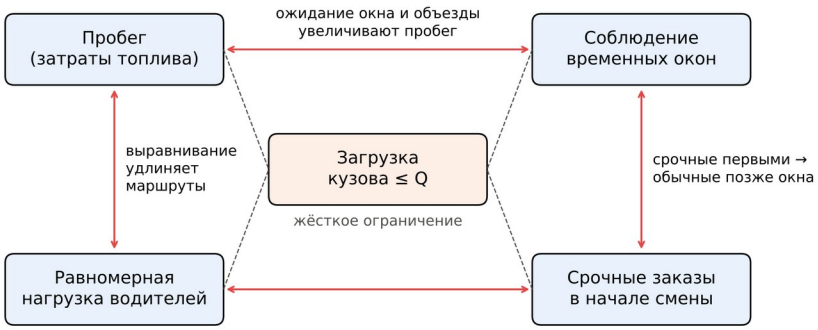


Рисунок 1 – Конфликт критериев качества плана доставки

Сравнение планов требует свёртки показателей в одну оценку. Взвешенная сумма предполагает постоянный «курс обмена» между критериями, тогда как логист рассуждает нелинейно: «если опозданий нет, лишние 5 % пробега допустимы; если опоздания массовые, экономия пробега план не спасает». Такие зависимости естественно описываются правилами «ЕСЛИ … ТО …» над нечёткими понятиями [5].

1.5 Лицо, принимающее решение, и необходимые ему зависимости

Лицом, принимающим решение, является диспетчер (логист) службы доставки: он утверждает план на смену и отвечает за затраты и своевременность доставки. Зависимости, которые он должен учитывать, приведены в таблице 2; именно они формализуются в базе нечётких правил (подраздел 3.3).

Таблица 2 – Зависимости, которые должен учитывать ЛПР

Ситуация	Последствие	Вывод для оценки плана
Пробег близок к минимальному, опозданий нет	Минимальные затраты при полном сервисе	Качество очень высокое
Пробег короткий, опоздания массовые	Экономия топлива ценой потери клиентов	Качество не выше среднего
Пробег большой, опозданий нет	Сервис обеспечен за счёт перерасхода	Качество среднее
Срочные заказы доставлены в конце смены	Риск отказа клиента и штрафов	Качество снижается
Хотя бы одна машина перегружена	План физически неосуществим	Качество очень низкое, план отвергается
Один водитель работает вдвое дольше другого	Сверхурочные и опоздания у перегруженного водителя	Качество снижается

Понятия «пробег близок к минимальному», «опоздания массовые» не имеют чётких границ, а пороговые значения произвольны и создают скачки оценки, что определяет выбор нечёткой логики (раздел 2).

1.6 Постановка задачи курсовой работы

Заданы склад, N клиентов (координаты, объёмы, срочность, временные окна) и K одинаковых машин вместимостью Q. Требуется построить план доставки без перегруза, имеющий максимальную оценку качества по пяти критериям таблицы 1; оценка должна формироваться по правилам, понятным ЛПР, а поиск – занимать секунды. Исходные данные генерируются программно с фиксированным seed и сохраняются в JSON.

2 Выбор и применение интеллектуального метода

2.1 Обзор подходов искусственного интеллекта

Современные интеллектуальные системы, как правило, гибридны и сочетают несколько подходов искусственного интеллекта [10]: экспертные системы, семантические и фреймовые модели, нейронные сети, нечёткую логику, анализ данных, эвристики, агентные системы, эволюционные алгоритмы [11, 12]. Задача подраздела 1.6 распадается на две подзадачи разной природы: **поиск** плана в огромном дискретном пространстве и **оценку** плана по противоречивым критериям на основе качественных знаний ЛПР. Применимость подходов к ним сопоставлена в таблице 3.

Таблица 3 – Применимость подходов искусственного интеллекта к задаче

Подход	Поиск плана	Оценка плана	Вывод
Продукционная экспертная система	Не ищет решения	Правила ЛПР выражаются естественно, но чёткие пороги дают скачки оценки	Частично
Семантические и фреймовые модели	Не предназначены для оптимизации	Описывают понятия, не дают числовой оценки	Не подходит
Нейронные сети	Нужна выборка оптимальных планов, которой нет	Нужна размеченная выборка; «чёрный ящик»	Не подходит
Нечёткая логика (Мамдани)	Не ищет решения	Формализует правила ЛПР; оценка плавная и объяснимая	Для оценки
Data mining	Кластеризация клиентов – лишь предварительный шаг	Не оценивает план	Вспомогательно
Эвристики	Быстры, но учитывают один критерий	Не оценивают	Базовый уровень
Агентные системы	Возможны (агент-машина), но сложны	Нужна общая функция оценки	Избыточно
Генетические алгоритмы	Эффективный поиск в пространстве перестановок с любой функцией качества	Требуют внешней функции приспособленности	Для поиска

Ни один подход не решает задачу целиком: ГА хорошо ищет, но «не знает», что такое хороший план, а нечёткая система оценивает, но не ищет. Нейросетевые методы и анализ данных [13] требуют выборок, которых нет, а агентные подходы, развитые в робототехнике [14], здесь избыточны.

2.2 Обоснование выбора методов и гибридной схемы

Почему генетический алгоритм, а не точный метод. Полный перебор невозможен уже при двадцати клиентах (подраздел 1.3), а точные методы требуют линейной целевой функции, тогда как оценка плана нелинейна и вычисляется алгоритмически. ГА не накладывает на целевую функцию требований – достаточно уметь вычислить её значение [2]; он работает с популяцией решений, естественно представляет план перестановкой и хорошо зарекомендовал себя в маршрутизации [9, 3]. Цена – отсутствие гарантии оптимальности (подраздел 4.3).

Почему нечёткая логика, а не взвешенная сумма. Линейная свёртка предполагает постоянную взаимозаменяемость критериев, тогда как зависимости ЛПР нелинейны (таблица 2); её веса трудно назначить и объяснить («сколько километров стоит одно опоздание?»), а результат не интерпретируется словами. Нечёткая система хранит знания в виде правил «ЕСЛИ протяжённость короткая И соблюдение окон хорошее ТО качество очень высокое», которые эксперт формулирует и проверяет непосредственно [5, 15]; плавные функции принадлежности устраняют скачки оценки, а протокол вывода показывает, какие правила определили оценку.

Гибридная схема. Известны схемы, в которых ГА настраивает параметры нечёткой системы по обучающей выборке, и схемы, в которых нечёткий вывод используется внутри ГА [16, 17]. Выбран второй вариант – **нечёткая функция приспособленности**: ГА ищет план, а система вывода Мамдани вычисляет его оценку $Q \in [0; 1]$, которая и является приспособленностью особи. Методы не независимы: на каждом поколении для каждой новой особи ГА декодирует хромосому, моделирует маршруты,

вычисляет пять показателей и вызывает нечёткий вывод; обучающая выборка не нужна. Жёсткое ограничение вместимости учитывается штрафом вне нечёткого блока (подраздел 3.4): нечёткая оценка «мягкая», и без штрафа перегруженный план с прекрасными остальными показателями мог бы оказаться лучше допустимого, а ГА сходился бы к неосуществимым решениям.

2.3 Генетические алгоритмы

Генетический алгоритм предложен Дж. Холландом [18] и развит Д. Голдбергом [2]: популяция решений (особей) эволюционирует, причём более приспособленные особи чаще становятся родителями. Основные элементы ГА [3, 16]:

кодирование решения хромосомой (для маршрутизации – перестановкой); **функция приспособленности** f; **отбор** родителей; **скрещивание**, передающее потомкам признаки двух родителей; **мутация**, поддерживающая разнообразие; **элитизм** – перенос лучших особей без изменений; **критерии останова** – число поколений, стагнация, лимит времени.

При **рулеточном** отборе вероятность выбора особи пропорциональна её приспособленности (P – размер популяции):

$$p_i = \frac{f_i}{\sum_{j=1}^{P} f_j} \tag{5}$$

При **ранговом** отборе вероятность пропорциональна номеру особи в упорядоченной популяции. При **турнирном** отборе родителем становится лучшая из k случайно выбранных особей; давление отбора задаётся k и не зависит от масштаба f [2]. Обычные операторы скрещивания неприменимы к перестановкам (клиент может повториться или потеряться), поэтому используется **упорядоченный кроссовер** OX Л. Дэвиса [19]: отрезок копируется из первого родителя, остальные гены – в порядке их следования

во втором. Мутации для перестановок – обмен двух генов, инверсия отрезка (аналог улучшения 2-opt) и перенос гена.

По теореме о схемах короткие высокоприспособленные фрагменты решений («строительные блоки») получают в следующих поколениях всё больше копий [18]. Однако ГА – стохастический метод и **не гарантирует нахождения глобального оптимума**; типичная проблема – преждевременная сходимость, которой противодействуют мутация, умеренное давление отбора и достаточный размер популяции.

2.4 Нечёткая логика и алгоритм вывода Мамдани

Теория нечётких множеств предложена Л. Заде [4]. Нечёткое множество A на универсуме X задаётся функцией принадлежности $\mu_A(x)$: $X \to [0; 1]$ – степенью, с которой x принадлежит A. Чаще всего используются кусочно-линейные функции [15] – треугольная ($a \le b \le c$) и трапециевидная ($a \le b \le c \le d$):

$$\mu(x) = max\left(0, min\left(\frac{x-a}{b-a}, \frac{c-x}{c-b}\right)\right) \tag{6}$$

$$\mu(x) = max\left(0, min\left(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c}\right)\right) \tag{7}$$

При $a = b$ (или $c = d$) получаются «плечевые» функции, равные единице на краю универсума. **Лингвистическая переменная** – переменная, значения которой (термы) – слова, заданные нечёткими множествами на общем универсуме, например «Протяжённость» с термами «короткая», «средняя», «длинная» [5]. Логическое «И» реализуется минимумом степеней принадлежности, «ИЛИ» – максимумом. Минимум – наиболее «осторожная» t-норма: истинность составного условия определяется самым слабым подусловием и не уменьшается искусственно с ростом числа подусловий, как при умножении.

Знания эксперта записываются в виде правил «ЕСЛИ условие ТО заключение» с весовыми коэффициентами $F \in (0; 1]$ – степенью уверенности

в правиле. Алгоритм Мамдани [20] включает шесть последовательных этапов [5, 17]:

1) **Формирование базы правил** вида $RULE_j$: ЕСЛИ «условие$_1$» И … И «условие$_m$» ТО «заключение» (F_j), j = 1, …, n.

2) **Фаззификация**: для чётких входов A = {a_1, …, a_m} вычисляются степени истинности всех k подусловий:

$$b_i = \mu_i(a_i), i = 1, \ldots, k \tag{8}$$

3) **Агрегирование подусловий**: степень истинности условия правила – минимум по его подусловиям I_j:

$$c_j = \min_{i \in I_j}(b_i), j = 1, \ldots, n \tag{9}$$

4) **Активизация**: степень истинности заключения умножается на вес правила, и функция принадлежности терма заключения «срезается» на этом уровне:

$$d_j = c_j \cdot F_j, \mu'_j(x) = \min(d_j, \mu_j(x)) \tag{10}$$

5) **Аккумуляция**: активизированные множества объединяются операцией max в множество E выходной переменной:

$$\mu_E(x) = \max_{j=1,\ldots,n}(\mu'_j(x)) \tag{11}$$

6) **Дефаззификация** методом центра тяжести (Min, Max – границы универсума выхода):

$$y = \frac{\int_{Min}^{Max} x \cdot \mu_E(x)\,dx}{\int_{Min}^{Max} \mu_E(x)\,dx} \tag{12}$$

Центр тяжести учитывает вклад всех сработавших правил, и результат меняется плавно – для функции приспособленности это важно: ГА узнаёт, не только какое решение лучше, но и насколько. Недостаток – зависимость результата от выбора функций принадлежности [5] – компенсируется экспериментальной проверкой (раздел 4).

3 Разработка интеллектуальной системы

3.1 Архитектура системы

Система реализована на Python 3 в виде набора модулей (рисунок 2); библиотека NumPy применяется только для векторных вычислений.



Рисунок 2 – Архитектура гибридной интеллектуальной системы

Модуль model.py содержит классы предметной области и моделирование маршрутов, generator.py – генератор задач, пакет fuzzy – функции принадлежности, лингвистические переменные, базу правил и вывод Мамдани, пакет ga – кодирование, операторы, функцию приспособленности и цикл ГА, baselines.py – базовые методы для сравнения (случайное решение, «ближайший сосед», полный перебор), модули visualize.py, gui.py и main.py – графики и интерфейсы. На этапе оценки приспособленности ГА передаёт хромосому функции приспособленности, та вычисляет пять показателей плана и вызывает нечёткий вывод, а полученная оценка Q возвращается в ГА и используется при отборе родителей. Диаграмма основных классов приведена в приложении Г.

Графический интерфейс (gui.py, tkinter) содержит панель параметров задачи и ГА, кнопки «Сгенерировать», «Загрузить/Сохранить JSON», «Решить», «Стоп», «Жадный алгоритм» и вкладки с картой маршрутов, графиком сходимости и результатом нечёткого вывода; ГА работает в отдельном потоке и обновляет карту каждые десять поколений. Консольный интерфейс (main.py) предоставляет те же возможности (пример вывода – в приложении В).

3.2 Модель задачи и генератор исходных данных

Задача описывается классом Instance (склад, клиенты, K, Q, скорость 0,5 км/мин, смена 480 мин). Функция simulate_route моделирует движение машины с ожиданием открытия окон и учётом опозданий по формуле (3) и разгрузкой 10 мин; evaluate_routes вычисляет показатели плана – входы нечёткой модели.

Генератор generate_instance(n_clients, n_vehicles, seed) размещает клиентов равномерно в квадрате 40 × 40 км со складом в центре, назначает объёмы заказов 1–10 единиц и срочность (60 % обычных, 25 % повышенной срочности, 15 % срочных); вместимость подбирается так, чтобы парк был загружен на 85 %. Чтобы решение без опозданий заведомо существовало, временные окна строятся вокруг моментов прибытия в скрытом опорном плане, построенном методом заметания (клиенты делятся по полярному углу на K секторов). Опорный план системе не сообщается. Пример файла задачи приведён в приложении Б.

3.3 Нечёткая модель оценки качества плана

3.3.1 Лингвистические переменные

Входные переменные вычисляются для плана в целом, а не по маршрутам с усреднением: сбалансированность по определению характеризует соотношение маршрутов, а суммарные показатели соответствуют тому, как план оценивает ЛПР. Чёткие значения входов:

– протяжённость – отношение суммарного пробега D к пробегу D_{ref} жадного решения «ближайший сосед» на той же задаче:

$$L = \frac{D}{D_{ref}} \tag{13}$$

– загрузка транспорта – максимальная загрузка машины W_k в долях вместимости:

$$3 = \max_k \frac{W_k}{Q} \tag{14}$$

– соблюдение временных окон – средняя по клиентам относительная величина опоздания, где опоздание на 60 мин и более считается полным:

$$T = \frac{1}{N} \sum_{i=1}^{N} min\left\{1, \frac{late_i}{60}\right\} \tag{15}$$

– обслуживание срочных заказов – средневзвешенное время начала обслуживания срочных клиентов в долях смены (вес $u_i - 1$ равен 0 для обычных, 1 для повышенной срочности и 2 для срочных заказов):

$$U = \frac{\sum_i (u_i - 1) \cdot \frac{s_i}{H}}{\sum_i (u_i - 1)} \tag{16}$$

– сбалансированность – коэффициент вариации длительностей маршрутов τ_k всех K машин (отношение среднеквадратического отклонения к среднему):

$$B = \frac{\sigma_\tau}{m_\tau} \tag{17}$$

Нормировка к эталонному решению делает протяжённость безразмерной: L = 1 – «как у жадного алгоритма», L = 0,9 – на 10 % короче. Выходная переменная «Качество решения» задана на [0; 1] пятью треугольными термами. Параметры всех функций принадлежности приведены в таблице 4, их графики – на рисунке 3.

Таблица 4 – Лингвистические переменные и функции принадлежности

Переменная	Универсум	Термы и параметры функций принадлежности
Протяжённость (L)	[0,6; 2,6]	«короткая» – Trap(0,60; 0,60; 0,80; 1,10); «средняя» – Tri(0,80; 1,10; 1,60); «длинная» – Trap(1,10; 1,60; 2,60; 2,60)
Загрузка транспорта (З)	[0,0; 2,0]	«низкая» – Trap(0,00; 0,00; 0,50; 0,80); «нормальная» – Trap(0,50; 0,80; 1,00; 1,10); «перегрузка» – Trap(1,00; 1,10; 2,00; 2,00)
Соблюдение временных окон (T)	[0,0; 1,0]	«хорошее» – Tri(0,00; 0,00; 0,12); «удовлетворительное» – Tri(0,00; 0,12; 0,45); «плохое» – Trap(0,12; 0,45; 1,00; 1,00)
Обслуживание срочных заказов (U)	[0,0; 1,0]	«хорошее» – Trap(0,00; 0,00; 0,05; 0,25); «среднее» – Tri(0,05; 0,25; 0,50); «плохое» – Trap(0,25; 0,50; 1,00; 1,00)
Сбалансированность (B)	[0,0; 1,5]	«равномерная» – Trap(0,00; 0,00; 0,05; 0,40); «неравномерная» – Trap(0,05; 0,40; 1,50; 1,50)
Качество решения (Q)	[0,0; 1,0]	«очень низкое» – Tri(0,00; 0,00; 0,25); «низкое» – Tri(0,00; 0,25; 0,50); «среднее» – Tri(0,25; 0,50; 0,75); «высокое» – Tri(0,50; 0,75; 1,00); «очень высокое» – Tri(0,75; 1,00; 1,00)

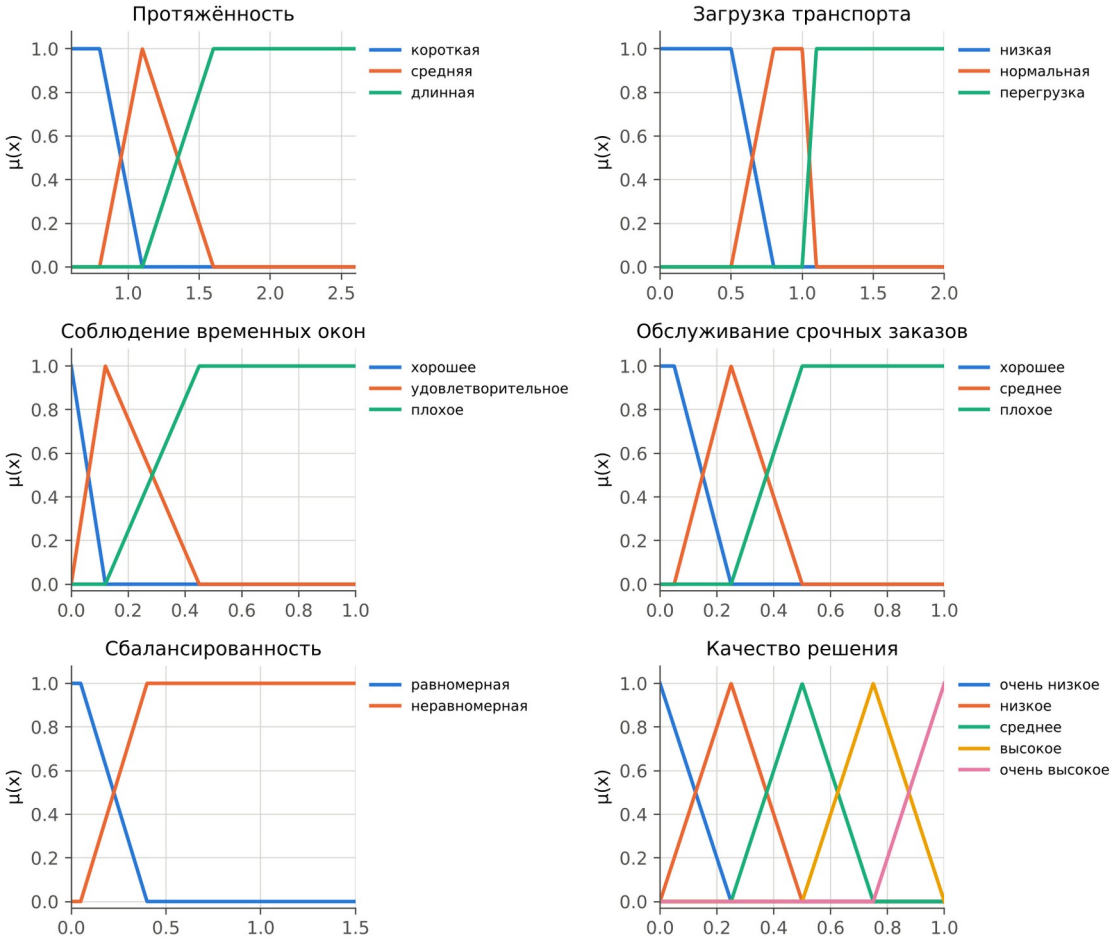


Рисунок 3 – Функции принадлежности лингвистических переменных

Соседние термы входных переменных образуют разбиение Руспини: склоны пересекаются на уровне 0,5, сумма степеней принадлежности равна

единице. Такой выбор сделан по результатам отладки: в первой версии модели трапеции с широкими плоскими вершинами создавали «плато» – если все сработавшие правила имеют одно заключение, центр тяжести не зависит от степеней их истинности, и улучшение плана не меняет оценку. При разбиении Руспини почти любое значение принадлежит двум термам, срабатывают правила с разными заключениями, и оценка меняется плавно.

3.3.2 База нечётких правил

База содержит 26 правил с весовыми коэффициентами (таблица 5; обозначения переменных: L – протяжённость, З – загрузка, Т – соблюдение окон, U – обслуживание срочных заказов, В – сбалансированность, Q – качество). Её ядро – правила 1–9 – образуют полную таблицу «Протяжённость × Соблюдение временных окон»: улучшение любого из двух главных показателей на один терм повышает заключение на один терм. Поскольку термы каждой переменной покрывают весь универсум, при любых входных данных срабатывает хотя бы одно из этих правил, и база правил полна. Правила 10–26 уточняют оценку с учётом срочных заказов, загрузки и сбалансированности. Положительные правила для вспомогательных критериев сформулированы только вместе с хорошим значением пробега или окон («ЕСЛИ срочные обслужены хорошо И опозданий нет ТО …»), чтобы хорошая сбалансированность не «вытягивала» план с массовыми опозданиями. Весовые коэффициенты $F < 1$ назначены правилам, в которых эксперт менее уверен или которые описывают второстепенные критерии.

Таблица 5 – База нечётких продукционных правил

№	Правило	F	Пояснение
1	ЕСЛИ L = короткая И Т = хорошее ТО Q = очень высокое	1,0	Короткий пробег без опозданий — главная цель планирования доставки.
2	ЕСЛИ L = короткая И Т = удовлетворительное ТО Q = высокое	1,0	Пробег экономичен, опоздания единичны и невелики.
3	ЕСЛИ L = короткая И Т = плохое ТО Q = среднее	1,0	Экономия пробега получена ценой массовых опозданий — компромиссный план.
4	ЕСЛИ L = средняя И Т = хорошее ТО Q = высокое	1,0	Опозданий нет, перерасход пробега умеренный.

№	Правило	F	Пояснение
5	ЕСЛИ L = средняя И T = удовлетворительное ТО Q = среднее	1,0	Средний пробег и отдельные опоздания — план посредственный.
6	ЕСЛИ L = средняя И T = плохое ТО Q = низкое	1,0	Пробег не сокращён, а клиенты обслуживаются с опозданием.
7	ЕСЛИ L = длинная И T = хорошее ТО Q = среднее	1,0	Окна соблюдены, но за счёт большого перерасхода топлива и времени.
8	ЕСЛИ L = длинная И T = удовлетворительное ТО Q = низкое	1,0	Длинные маршруты не спасают от опозданий — план неэффективен.
9	ЕСЛИ L = длинная И T = плохое ТО Q = очень низкое	1,0	Большой пробег и массовые опоздания одновременно — план следует отвергнуть.
10	ЕСЛИ U = хорошее И T = хорошее ТО Q = очень высокое	0,8	Срочные клиенты обслужены в начале смены, опозданий нет.
11	ЕСЛИ U = хорошее И L = короткая ТО Q = высокое	0,7	Срочные заказы доставлены первыми без перерасхода пробега.
12	ЕСЛИ U = среднее И T = хорошее ТО Q = высокое	0,6	Опозданий нет, срочные заказы доставлены не первыми, но вовремя.
13	ЕСЛИ U = среднее И T = удовлетворительное ТО Q = среднее	0,6	Посредственное обслуживание и срочных, и обычных клиентов.
14	ЕСЛИ U = плохое ТО Q = низкое	0,8	Срочные заказы доставляются в конце смены — риск потери клиентов.
15	ЕСЛИ U = плохое И T = плохое ТО Q = очень низкое	1,0	Опоздания вместе с поздней доставкой срочных заказов — худший сервис.
16	ЕСЛИ З = перегрузка ТО Q = очень низкое	1,0	Перегруз машины физически недопустим (дополнительно штрафуется вне нечёткого блока).
17	ЕСЛИ З = нормальная И L = короткая И T = хорошее ТО Q = очень высокое	0,8	Рациональная загрузка кузова при коротком пробеге и без опозданий.
18	ЕСЛИ З = низкая И L = длинная ТО Q = низкое	0,8	Машины недогружены, но много ездят — парк используется неэффективно.
19	ЕСЛИ В = равномерная И T = хорошее ТО Q = высокое	0,7	Нагрузка водителей равномерна, сверхурочных нет, клиенты обслужены вовремя.
20	ЕСЛИ В = равномерная И L = короткая ТО Q = высокое	0,6	Равные по длительности маршруты при экономичном пробеге.
21	ЕСЛИ В = неравномерная ТО Q = низкое	0,4	Один водитель перегружен, другие простаивают.
22	ЕСЛИ В = неравномерная И T = удовлетворительное ТО Q = низкое	0,7	Перегруженный водитель опаздывает, пока другие простаивают.
23	ЕСЛИ В = неравномерная И L = средняя ТО Q = низкое	0,6	Неравномерная нагрузка при среднем пробеге — признак неудачного распределения заказов.
24	ЕСЛИ L = короткая И U = хорошее И В = равномерная ТО Q = очень высокое	0,9	Экономичный план с приоритетом срочных заказов и равной нагрузкой водителей.
25	ЕСЛИ L = длинная И U = плохое ТО Q = очень низкое	0,8	Длинные маршруты, на которых срочные заказы доставляются последними.
26	ЕСЛИ L = средняя И U = хорошее ТО Q = высокое	0,6	Небольшой перерасход пробега оправдан ранней доставкой срочных заказов.

3.3.3 Реализация механизма вывода

Алгоритм вывода реализован классом MamdaniInference в точном соответствии с шестью этапами подраздела 2.4 (рисунок 4). Универсум выходной переменной дискретизирован равномерной сеткой из G = 201 точки, поэтому интегралы в формуле (12) заменены суммами:

$$Q \approx \frac{\sum_{g=1}^{G} x_g \cdot \mu_E(x_g)}{\sum_{g=1}^{G} \mu_E(x_g)} \tag{18}$$

Функции принадлежности термов-заключений вычисляются на сетке один раз и хранятся в матрице n × G, что позволяет активизировать все правила одной векторной операцией (листинг 1).

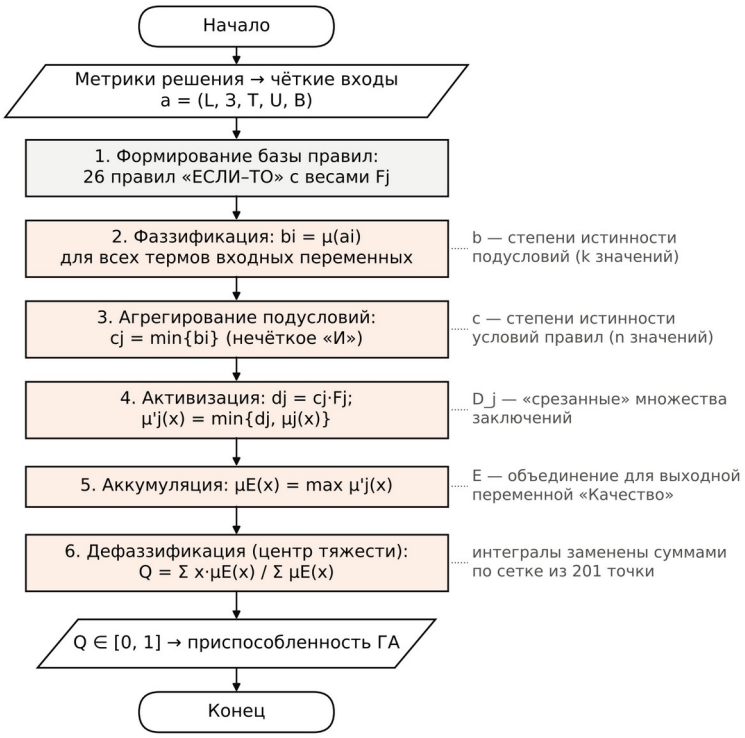


Рисунок 4 – Схема алгоритма нечёткого вывода Мамдани

Листинг 1 – Этапы агрегирования, активизации, аккумуляции и дефаззификации

```
def aggregate(self, fuzz: dict[str, dict[str, float]]) -> np.ndarray:
    # c_j = min{b_i} по всем подусловиям (переменная v, терм t) правила j
    return np.array([min(fuzz[v][t] for v, t in r.conditions) for r in self.rules])

def activate(self, c: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    d = c * self._weights                                   # d_j = c_j · F_j
    # μ'_j(x) = min(d_j, μ_j(x)): строка j — «срезанный» терм заключения правила j
    activated = np.minimum(d[:, None], self._concl_mf)
    return d, activated

@staticmethod
def accumulate(activated: np.ndarray) -> np.ndarray:
    return activated.max(axis=0)                            # μ_E(x) = max_j μ'_j(x)

def defuzzify(self, mu: np.ndarray) -> float:
    area = mu.sum()                                         # Σ μ_E(x_g)
    if area <= 1e-12:
        return 0.0
```

```
    return float((self.grid * mu).sum() / area)           # Σ x_g·μ_E(x_g) / Σ
μ_E(x_g)
```

Метод trace сохраняет все промежуточные результаты, что позволяет показать ЛПР, какие правила определили оценку (подраздел 4.6).

3.4 Генетический алгоритм

3.4.1 Кодирование решения

Решение кодируется перестановкой длины N + K − 1: гены 1, …, N – номера клиентов, гены N + 1, …, N + K − 1 – разделители маршрутов (рисунок 5); хромосома одновременно задаёт распределение заказов по машинам и порядок объезда. Кодирование «гигантский тур с разбиением по вместимости» [9] отклонено: оно может потребовать больше K машин и лишает смысла переменную «Загрузка». Перестановка позволяет применять стандартный OX: его потомок – всегда корректная перестановка.



Рисунок 5 – Кодирование плана доставки и упорядоченное скрещивание

3.4.2 Функция приспособленности

Приспособленность особи x равна нечёткой оценке её качества за вычетом штрафа за перегруз:

$$f(x)=Q(x)-P(x),\ P(x)=\begin{cases}0, \text{если перегруза нет};\\ 1+\dfrac{\Delta}{Q}, \text{если } \Delta>0,\end{cases} \tag{19}$$

где Δ – суммарное превышение вместимости по всем машинам. Поскольку $Q \in [0; 1]$, любой план с перегрузом получает отрицательную приспособленность и оказывается хуже любого допустимого плана; слагаемое Δ/Q сохраняет различия между недопустимыми планами, чтобы ГА мог постепенно выводить популяцию из недопустимой области. Вычисленные значения кэшируются по хромосоме: одинаковые особи, часто возникающие после элитизма, повторно не оцениваются.

3.4.3 Генетические операторы

Отбор родителей – турнирный с размером турнира k = 3 (для сравнения реализованы также рулеточный и ранговый отбор). Скрещивание OX выполняется с вероятностью p_c = 0,9 и даёт двух потомков (родители меняются ролями); реализация приведена в листинге 2. Каждый потомок с вероятностью p_m подвергается одной из трёх мутаций, выбранной равновероятно:

– обмен двух случайных генов – меняет местами клиентов (возможно, из разных маршрутов) или перемещает разделитель;

– инверсия отрезка – разворачивает часть маршрута, что устраняет самопересечения (аналог улучшения 2-opt);

– перенос клиента – удаляет случайного клиента из его маршрута и вставляет в случайную позицию другого маршрута, изменяя распределение заказов.

Листинг 2 – Оператор упорядоченного скрещивания OX

```
def order_crossover(p1: list[int], p2: list[int], rng: np.random.Generator) ->
list[int]:
    n = len(p1)
    i, j = sorted(rng.choice(n + 1, size=2, replace=False))
    child = [None] * n
    child[i:j] = p1[i:j]
    taken = set(p1[i:j])
    fill = [p2[(j + k) % n] for k in range(n) if p2[(j + k) % n] not in taken]
    pos = j % n
    for g in fill:
        while child[pos] is not None:
            pos = (pos + 1) % n
        child[pos] = g
        pos = (pos + 1) % n
    return child
```

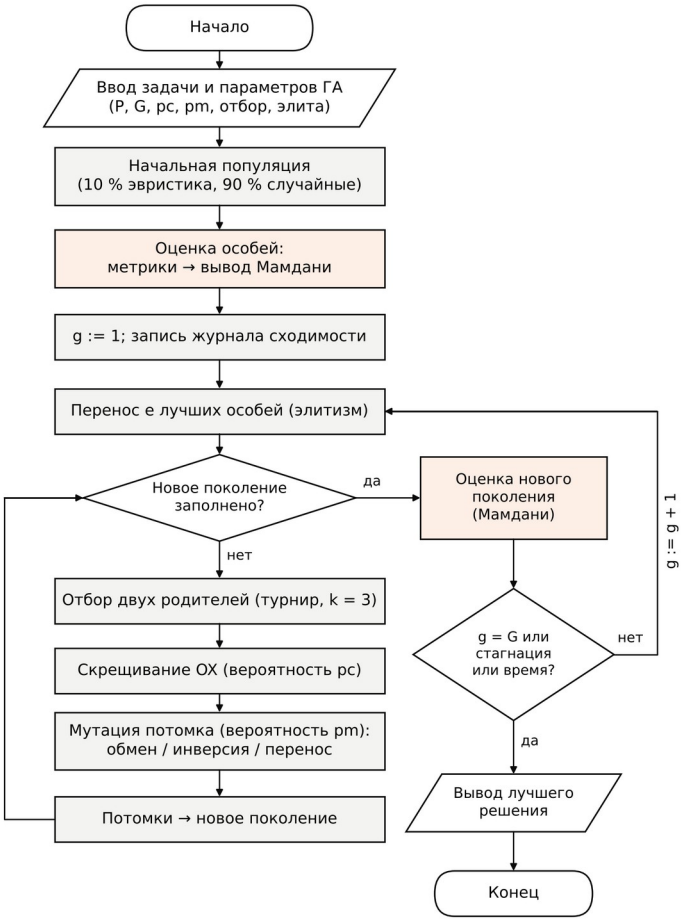


Рисунок 6 – Схема генетического алгоритма

3.4.4 Начальная популяция, элитизм и останов

Схема алгоритма приведена на рисунке 6. Начальная популяция на 90 % состоит из случайных перестановок и на 10 % – из решений рандомизированного «ближайшего соседа» (следующий клиент выбирается случайно среди трёх ближайших). «Засев» эвристикой – распространённый приём [9]: без него при N = 50 все случайные особи попадают в область, где нечёткая оценка постоянна. Две лучшие особи переносятся в следующее поколение без изменений (элитизм). Алгоритм останавливается по достижении заданного числа поколений, при отсутствии улучшения в течение 120 поколений (стагнация) или по лимиту времени. Параметры по умолчанию (P = 100, p_c = 0,9, p_m = 0,2, k = 3, две элиты) выбраны по экспериментам подраздела 4.5.

4 Экспериментальный анализ

4.1 Методика экспериментов

Все эксперименты выполняются одной командой (скрипт experiments/run_experiments.py, около 85 с на четырёхъядерном процессоре). Задачи и алгоритмы используют фиксированные seed, поэтому повторный запуск даёт те же значения приспособленности (время работы может немного отличаться).

Для каждого размера N ∈ {10, 20, 30, 50} сгенерировано пять задач (число машин K = 2, 3, 4 и 7 соответственно). Сравниваются четыре метода:

– случайное решение – средние показатели 20 случайных хромосом;

– жадный алгоритм «ближайший сосед» с учётом вместимости;

– ГА со взвешенной суммой тех же нормированных показателей вместо нечёткого блока (веса 0,35; 0,30; 0,20; 0,15 для протяжённости, окон, срочности и сбалансированности) и тем же штрафом за перегруз:

$$Q_{ws} = 1 - \left(0{,}35 \cdot s_L + 0{,}30 \cdot s_T + 0{,}20 \cdot s_U + 0{,}15 \cdot s_B\right) \qquad (20)$$

– предлагаемый ГА с нечёткой оценкой (P = 100, 300 поколений при N ≤ 30 и 500 при N = 50, останов по стагнации 150 поколений).

Все планы оцениваются одной нечёткой моделью Q и «сырыми» показателями.

4.2 Сравнение с базовыми методами

Результаты (средние по пяти задачам) приведены в таблице 6 и на рисунке 7.

Таблица 6 – Сравнение методов (среднее ± СКО по пяти задачам)

N	Метод	Пробег, км	Число опозданий	Опоздание, мин	Перегруз	B	U	Оценка Q	Время, с
10	Случайное	221 ± 20	5,5	859	11,0	0,49	0,40	0,263 ± 0,020	0,00
10	Ближайший сосед	169 ± 21	2,0	190	0,0	0,16	0,19	0,548 ± 0,105	0,00
10	ГА + взвеш. сумма	158 ± 13	0,2	13	0,0	0,06	0,13	0,736 ± 0,054	0,34
10	ГА + нечёткая оценка	164 ± 24	0,0	0	0,0	0,03	0,13	0,780 ± 0,009	0,40

N	Метод	Пробег, км	Число опозданий	Опоздание, мин	Перегруз	B	U	Оценка Q	Время, с
20	Случайное	438 ± 30	13,7	3114	25,3	0,62	0,52	0,201 ± 0,002	0,00
20	Ближайший сосед	247 ± 25	7,4	417	0,0	0,14	0,21	0,471 ± 0,043	0,00
20	ГА + взвеш. сумма	224 ± 26	2,6	34	0,0	0,09	0,14	0,680 ± 0,052	0,85
20	ГА + нечёткая оценка	237 ± 25	2,4	138	0,0	0,06	0,17	0,684 ± 0,077	0,99
30	Случайное	644 ± 33	21,3	6421	47,6	0,73	0,58	0,203 ± 0,001	0,00
30	Ближайший сосед	328 ± 29	13,6	1046	0,0	0,30	0,28	0,428 ± 0,053	0,00
30	ГА + взвеш. сумма	285 ± 15	4,2	64	0,0	0,05	0,14	0,712 ± 0,046	1,09
30	ГА + нечёткая оценка	313 ± 24	4,8	138	0,0	0,05	0,14	0,720 ± 0,037	1,36
50	Случайное	1097 ± 31	35,6	12033	81,8	0,84	0,59	0,204 ± 0,001	0,00
50	Ближайший сосед	461 ± 22	20,6	1824	0,0	0,49	0,29	0,438 ± 0,070	0,00
50	ГА + взвеш. сумма	442 ± 20	4,4	79	0,0	0,10	0,14	0,671 ± 0,027	2,20
50	ГА + нечёткая оценка	517 ± 8	5,4	96	0,0	0,08	0,14	0,683 ± 0,044	2,81

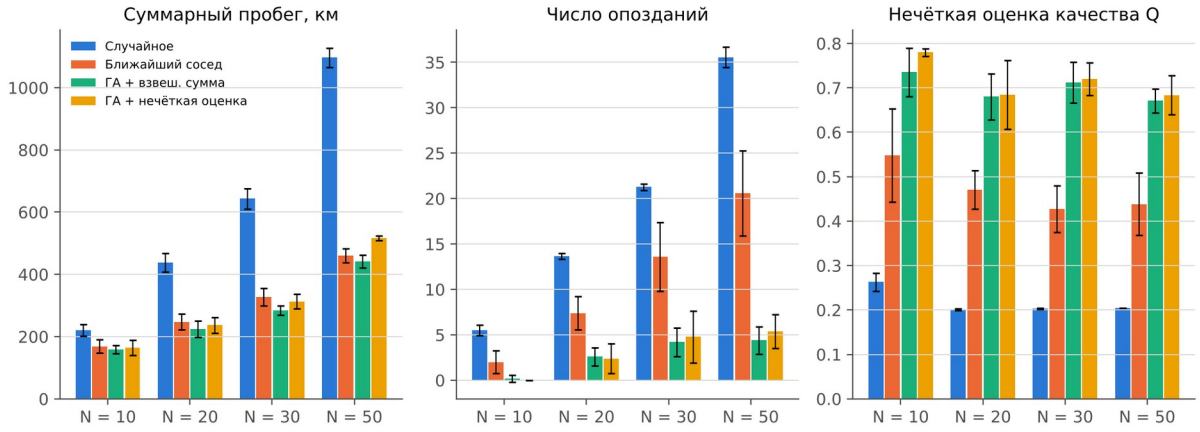


Рисунок 7 – Пробег, число опозданий и нечёткая оценка качества для четырёх методов

Выводы по эксперименту:

– оба варианта ГА многократно превосходят случайное решение и жадный алгоритм по соблюдению временных окон: при N = 50 суммарное опоздание сокращается с 1824 мин у «ближайшего соседа» до 96 мин у ГА с нечёткой оценкой (на 95 %), число опоздавших клиентов – с 20,6 до 5,4;

– при N ≤ 30 ГА с нечёткой оценкой одновременно сокращает и пробег (на 3–5 % относительно жадного алгоритма), то есть строит планы, лучшие по всем критериям; при N = 50 пробег возрастает на 12,2 % – это цена за

практически полное устранение опозданий и выравнивание нагрузки водителей (B = 0,08 против 0,49);

– нечёткая оценка вырастает на 0,21–0,29 по сравнению с жадным алгоритмом; перегрузов в решениях ГА нет – штраф надёжно удерживает поиск в допустимой области;

– ГА со взвешенной суммой даёт сопоставимые результаты: по нечёткой оценке Q он немного уступает (например, 0,671 против 0,683 при N = 50), а по пробегу выигрывает, поскольку вес пробега в свёртке наибольший. Различие отражает разные «предпочтения» двух моделей оценки: нечёткая модель по правилам 19–24 сильнее ценит равномерную нагрузку водителей и не позволяет экономии пробега компенсировать опоздания. Преимущество нечёткой модели, таким образом, не в «более высоком числе», а в том, что её предпочтения заданы явными, проверяемыми экспертом правилами, а оценка объясняется протоколом вывода (подраздел 4.6);

– время работы ГА – от 0,40 до 2,81 с, что приемлемо для ежедневного планирования.

4.3 Сравнение с полным перебором

Для проверки того, насколько близко ГА подходит к оптимуму, сгенерированы задачи с N = 5…8 клиентами и K = 2 машинами, для которых функция приспособленности максимизирована полным перебором всех различных планов (с точностью до перенумерации машин). ГА с параметрами по умолчанию запускался десять раз с разными seed (таблица 7).

Таблица 7 – Сравнение ГА с полным перебором (10 запусков ГА)

N	Число решений	Оптимум f*	Время перебора, с	ГА: средняя f	ГА: худшая f	Найден оптимум, %	Отставание, %	Уникальных оценок ГА	Время ГА, с
5	360	0,7931	0,02	0,7931	0,7931	100	0,0	297	0,24
6	2 520	0,6720	0,17	0,6720	0,6720	100	0,0	651	0,31
7	20 160	0,7692	1,37	0,7508	0,6939	60	2,4	818	0,33

N	Число решений	Оптимум f*	Время перебора, с	ГА: средняя f	ГА: худшая f	Найден оптимум, %	Отставание, %	Уникальных оценок ГА	Время ГА, с
8	181 440	0,7049	10,79	0,6924	0,6145	80	1,8	1223	0,39

При N = 5 и 6 ГА находит точный оптимум во всех запусках. При N = 7 и 8 оптимум найден в 60 % и 80 % запусков соответственно, а среднее отставание от оптимума составляет 2,4 % и 1,8 %. При N = 8 ГА вычисляет приспособленность лишь 1223 уникальных особей – 0,7 % от 181 440 вариантов полного перебора – и работает примерно в 27 раз быстрее перебора; с ростом N это соотношение быстро растёт в пользу ГА (подраздел 1.3). Это подтверждает положение подраздела 2.3: ГА находит оптимальное или близкое к нему решение, но не гарантирует оптимума в каждом запуске; для надёжности ГА можно запускать несколько раз с разными seed.

4.4 Сходимость генетического алгоритма

На рисунке 8 показана динамика приспособленности для задачи N = 30. В начальной популяции средняя приспособленность отрицательна (большинство случайных особей перегружены), но за 10–15 поколений популяция переходит в допустимую область. Лучшая приспособленность растёт ступенчато от 0,342 до 0,772 (0,553 к 50-му поколению); дальнейший рост связан с устранением последних опозданий. Разрыв между лучшей и средней приспособленностью сохраняется – мутации поддерживают разнообразие, преждевременной сходимости нет.

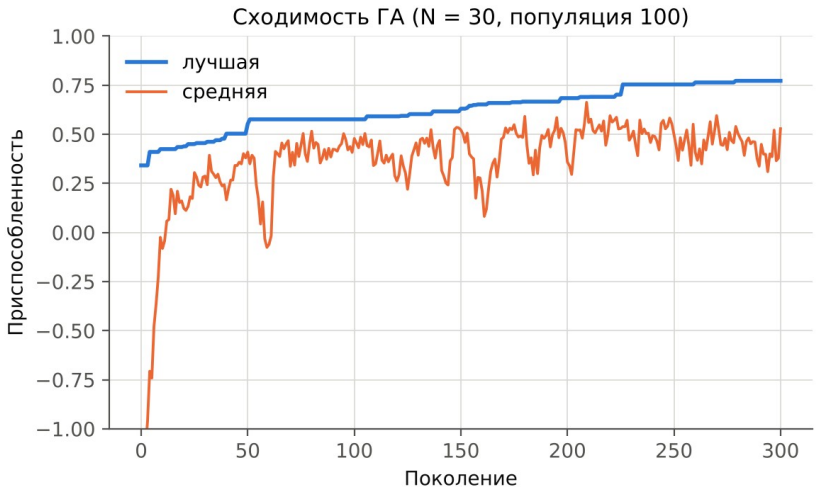


Рисунок 8 – Сходимость ГА: лучшая и средняя приспособленность (N = 30)

4.5 Влияние параметров генетического алгоритма

Исследовано влияние размера популяции, вероятности мутации и типа отбора на итоговую приспособленность при фиксированном числе поколений (300, без останова по стагнации). Каждая конфигурация запускалась на восьми задачах с N = 30; остальные параметры – по умолчанию. Средние значения итоговой приспособленности и их разброс показаны на рисунке 9 (полная таблица – файл experiments/results/exp4_params.csv).

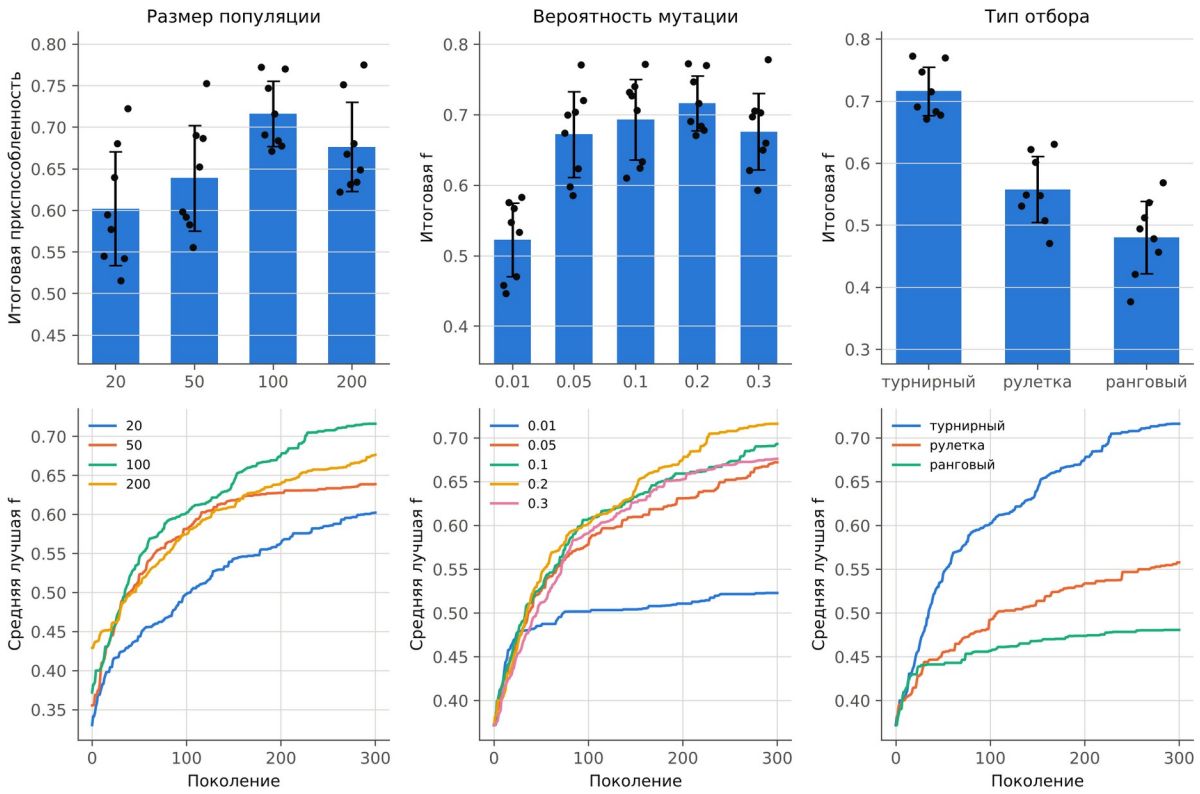


Рисунок 9 – Влияние параметров ГА: итоговая приспособленность (вверху) и средние кривые сходимости (внизу)

Выводы по эксперименту:

– **размер популяции**: при P = 20 и 50 не хватает разнообразия (0,602 и 0,639), лучший результат – при P = 100 (0,716); при P = 200 (0,676) лучшие решения распространяются медленнее, за 300 поколений поиск не завершается, а время удваивается;

– **вероятность мутации**: при p_m = 0,01 – преждевременная сходимость (0,523); с ростом p_m результат улучшается до 0,716 при 0,2, а при 0,3 снижается (0,676) – частые мутации разрушают удачные фрагменты маршрутов;

– **тип отбора**: турнирный (0,716) значительно лучше рулеточного (0,558) и рангового (0,481): приспособленности лежат в узком диапазоне 0,4–0,8, поэтому рулетка выбирает родителей почти равновероятно, а линейный ранговый отбор при P = 100 тоже создаёт слабое давление; турнир не зависит от масштаба значений.

Разброс между задачами (СКО 0,04–0,07) сопоставим с разницей близких конфигураций, поэтому выводы относятся к тенденциям.

4.6 Пример работы системы и протокол нечёткого вывода

На рисунке 10 показаны карты маршрутов для задачи N = 30: лучшая особь начального поколения и итоговое решение ГА (красные кольца – опоздания). Для сравнения, жадный алгоритм на этой задаче опаздывает к 11 клиентам при пробеге 304,4 км, а итоговое решение ГА короче (273,6 км) и содержит одно опоздание на 0,3 мин.

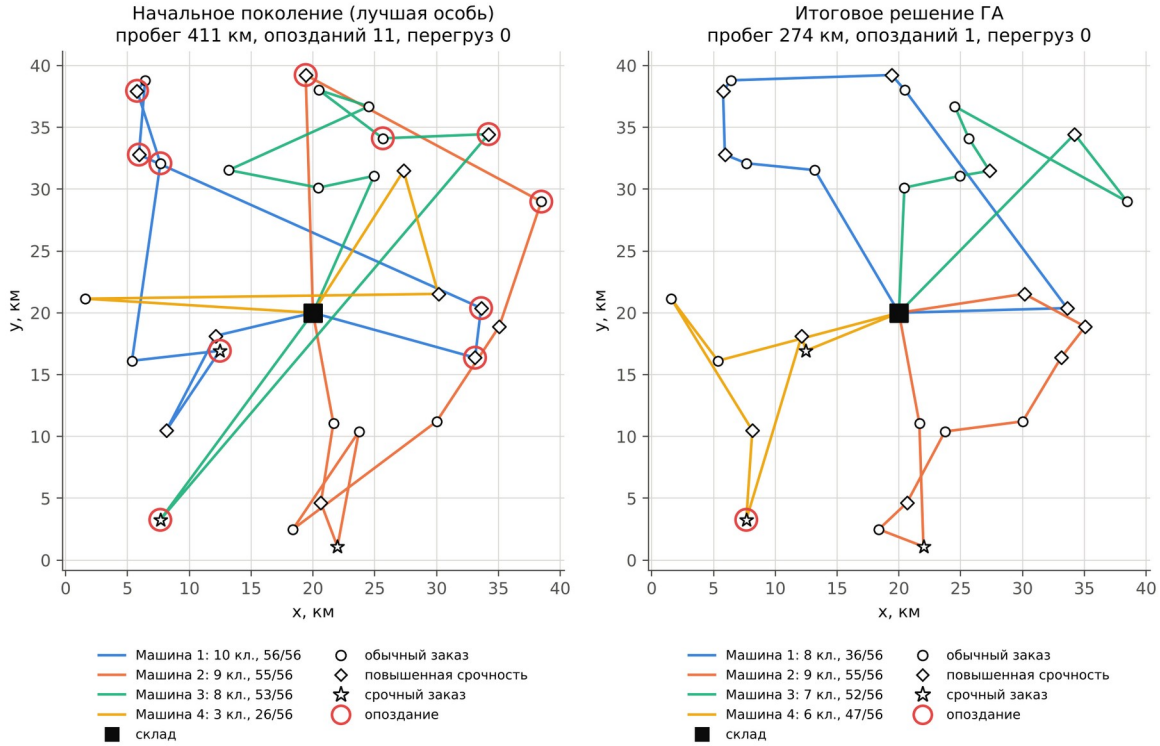


Рисунок 10 – Карты маршрутов: начальное поколение и итоговое решение ГА (N = 30)

Для итогового решения входы равны L = 0,899, З = 0,982, Т = 0,000, U = 0,197, В = 0,060. Фаззификация: L – «короткая» (0,671) и «средняя» (0,329); З – «нормальная» (1); Т – «хорошее» (0,999); U – «хорошее» (0,263) и «среднее» (0,737); В – «равномерная» (0,973).

В таблице 8 приведены правила с $d \geq 0{,}01$: подусловия, результат агрегирования c_j, вес и степень истинности заключения $d_j = c_j \cdot F_j$. Наибольший вклад вносят правила П19 (d = 0,681), П1 (d = 0,671) и П17 (d = 0,537).

Таблица 8 – Сработавшие правила и степени истинности

Правило	Подусловия и их степени истинности b	c = min b	F	d = c·F	Заключение (терм «Качества»)
П1	L=короткая: 0,671; T=хорошее: 0,999	0,671	1,0	0,671	очень высокое
П4	L=средняя: 0,329; T=хорошее: 0,999	0,329	1,0	0,329	высокое
П10	U=хорошее: 0,263; T=хорошее: 0,999	0,263	0,8	0,210	очень высокое
П11	U=хорошее: 0,263; L=короткая: 0,671	0,263	0,7	0,184	высокое
П12	U=среднее: 0,737; T=хорошее: 0,999	0,737	0,6	0,442	высокое
П17	З=нормальная: 1,000; L=короткая: 0,671; T=хорошее: 0,999	0,671	0,8	0,537	очень высокое
П19	B=равномерная: 0,973; T=хорошее: 0,999	0,973	0,7	0,681	высокое
П20	B=равномерная: 0,973; L=короткая: 0,671	0,671	0,6	0,403	высокое
П21	B=неравномерная: 0,027	0,028	0,4	0,011	низкое
П23	B=неравномерная: 0,027; L=средняя: 0,329	0,028	0,6	0,017	низкое
П24	L=короткая: 0,671; U=хорошее: 0,263; B=равномерная: 0,973	0,263	0,9	0,237	очень высокое
П26	L=средняя: 0,329; U=хорошее: 0,263	0,263	0,6	0,158	высокое

После аккумуляции каждый терм выходной переменной «срезан» на уровне максимальной степени истинности правил с этим заключением: «очень высокое» – 0,671, «высокое» – 0,681, «среднее» – 0,001, «низкое» – 0,016, «очень низкое» – 0,000. Множество сосредоточено на термах «высокое» и «очень высокое» (рисунок 11), дефаззификация даёт Q = 0,772. ЛПР получает и обоснование: план короче эталонного, почти без опозданий и равномерно загружает водителей, а оценку снижает то, что срочные заказы доставлены не в самом начале смены (U – «среднее» со степенью 0,737).

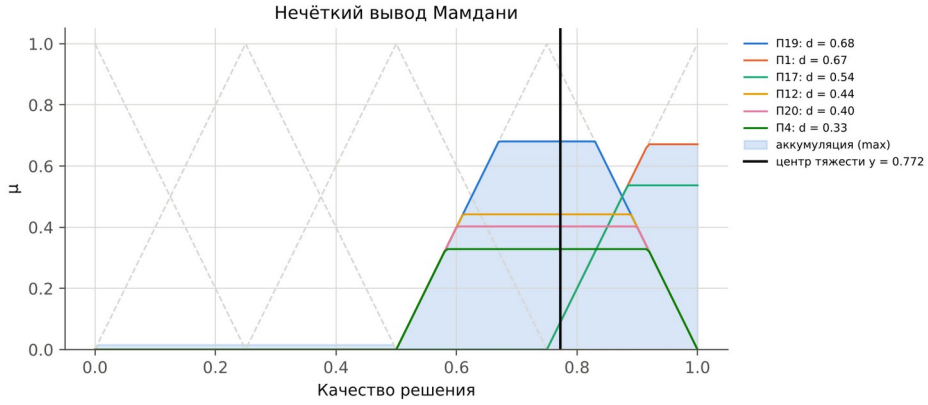


Рисунок 11 – Активизированные множества, аккумуляция и центр тяжести для итогового решения

4.7 Работа программы

[СКРИНШОТ: окно программы с решённой задачей на 30 клиентов (вкладка «Маршруты»)]

[СКРИНШОТ: вкладки «Сходимость» и «Нечёткий вывод»]

ЗАКЛЮЧЕНИЕ

Проделанная в рамках курсовой работы деятельность позволила ответить на поставленные вопросы, цели и задачи исследования. Основными задачами исследования являлись:

– анализ предметной области, постановка задачи, выявление критериев качества плана и зависимостей, учитываемых ЛПР;

– обоснование выбора генетического алгоритма для поиска плана, нечёткой логики для его оценки и гибридной схемы их взаимодействия;

– разработка нечёткой модели оценки плана, генетического алгоритма и программной системы;

– экспериментальный анализ системы.

Для решения этих задач были применены следующие методы:

– генетический алгоритм с перестановочным кодированием, турнирным отбором, скрещиванием OX, тремя мутациями и элитизмом;

– нечёткая логика: пять входных и одна выходная лингвистические переменные, база из 26 правил с весовыми коэффициентами, алгоритм Мамдани с дефаззификацией по центру тяжести;

– имитационное моделирование маршрутов, штрафные функции, вычислительный эксперимент.

В итоге выполнения курсовой работы удалось получить следующий результат решения конкретной проблемы в рамках выбранной предметной области: разработана гибридная система, в которой генетический алгоритм ищет распределение заказов по машинам и порядок объезда, а нечёткий вывод Мамдани оценивает каждую особь каждого поколения. Система строит допустимые планы за 0,4–2,5 с; при 50 клиентах она сокращает суммарное опоздание относительно жадного алгоритма на 95 %, а при 10–30 клиентах одновременно сокращает и пробег на 3–5 %. При $N \leq 6$ во всех запусках найден точный оптимум (проверено полным перебором), при $N = 7…8$ среднее отставание от оптимума не превышает 2,5 %.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Lenstra, J. K. Complexity of vehicle routing and scheduling problems / J. K. Lenstra, A. H. G. Rinnooy Kan // Networks. – 1981. – Vol. 11, № 2. – P. 221–227.

2. Goldberg, D. E. Genetic Algorithms in Search, Optimization, and Machine Learning / D. E. Goldberg. – Reading : Addison-Wesley, 1989. – 412 p.

3. Гладков, Л. А. Генетические алгоритмы / Л. А. Гладков, В. В. Курейчик, В. М. Курейчик ; под ред. В. М. Курейчика. – 2-е изд., испр. и доп. – М. : Физматлит, 2006. – 320 с.

4. Zadeh, L. A. Fuzzy sets / L. A. Zadeh // Information and Control. – 1965. – Vol. 8, № 3. – P. 338–353.

5. Леоненков, А. В. Нечеткое моделирование в среде MATLAB и fuzzyTECH / А. В. Леоненков. – СПб. : БХВ-Петербург, 2005. – 736 с.

6. Dantzig, G. B. The truck dispatching problem / G. B. Dantzig, J. H. Ramser // Management Science. – 1959. – Vol. 6, № 1. – P. 80–91.

7. Solomon, M. M. Algorithms for the vehicle routing and scheduling problems with time window constraints / M. M. Solomon // Operations Research. – 1987. – Vol. 35, № 2. – P. 254–265.

8. The Vehicle Routing Problem / ed. by P. Toth, D. Vigo. – Philadelphia : SIAM, 2002. – 367 p.

9. Prins, C. A simple and effective evolutionary algorithm for the vehicle routing problem / C. Prins // Computers & Operations Research. – 2004. – Vol. 31, № 12. – P. 1985–2002.

10. Джонс, М. Т. Программирование искусственного интеллекта в приложениях / М. Т. Джонс ; пер. с англ. А. И. Осипова. – М. : ДМК Пресс, 2011. – 312 с.

11. Боровская, Е. В. Основы искусственного интеллекта [Электронный ресурс] / Е. В. Боровская, Н. А. Давыдова. – 3-е изд. (эл.). – М. : Лаборатория знаний, 2016. – 130 с. – Режим доступа: http://znanium.com/catalog/product/366789 (дата обращения: 04.11.2026).

12. Жданов, А. А. Автономный искусственный интеллект [Электронный ресурс] / А. А. Жданов. – М. : БИНОМ. Лаборатория знаний, 2012. – Режим доступа: http://znanium.com/bookread2.php?book=539035 (дата обращения: 04.11.2026).

13. Чубукова, И. А. Data Mining / И. А. Чубукова. – М. : Интернет-университет информационных технологий : БИНОМ. Лаборатория знаний, 2006. – 382 с.

14. Станкевич, Л. А. Искусственный интеллект и искусственный разум в робототехнике : учеб. пособие / Л. А. Станкевич, Е. И. Юревич. – СПб. : Изд-во Политехн. ун-та, 2012. – 167 с.

15. Штовба, С. Д. Проектирование нечетких систем средствами MATLAB / С. Д. Штовба. – М. : Горячая линия – Телеком, 2007. – 288 с.

16. Рутковская, Д. Нейронные сети, генетические алгоритмы и нечеткие системы / Д. Рутковская, М. Пилиньский, Л. Рутковский ; пер. с польск. И. Д. Рудинского. – М. : Горячая линия – Телеком, 2006. – 452 с.

17. Пегат, А. Нечеткое моделирование и управление / А. Пегат ; пер. с англ. – М. : БИНОМ. Лаборатория знаний, 2009. – 798 с.

18. Holland, J. H. Adaptation in Natural and Artificial Systems / J. H. Holland. – Ann Arbor : University of Michigan Press, 1975. – 183 p.

19. Davis, L. Applying adaptive algorithms to epistatic domains / L. Davis // Proceedings of the 9th International Joint Conference on Artificial Intelligence. – Los Angeles, 1985. – Vol. 1. – P. 162–164.

20. Mamdani, E. H. An experiment in linguistic synthesis with a fuzzy logic controller / E. H. Mamdani, S. Assilian // International Journal of Man-Machine Studies. – 1975. – Vol. 7, № 1. – P. 1–13.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг основных модулей программы

Ниже приведены исходные тексты модулей, реализующих интеллектуальное ядро системы. Полный исходный код, включая графический интерфейс, визуализацию и тесты, находится в каталогах src и tests электронного приложения к работе.

Листинг А.1 – Модуль src/model.py – модель задачи и показатели плана

```
"""Модель задачи маршрутизации транспорта с временными окнами (CVRPTW).

Содержит классы предметной области (клиент, транспортное средство, задача,
решение), расчёт матрицы расстояний, моделирование движения машины по
маршруту и вычисление метрик решения, которые затем подаются на вход
нечёткого блока оценки качества.
"""
from __future__ import annotations

import json
import math
from dataclasses import dataclass, field, asdict
from pathlib import Path

import numpy as np


@dataclass
class Client:
    """Клиент (точка доставки).

    Атрибуты:
        id: номер клиента (1..N; 0 зарезервирован за складом).
        x, y: координаты на плоскости, км.
        demand: объём заказа, условные единицы груза.
        urgency: срочность заказа: 1 — обычный, 2 — повышенный, 3 — срочный.
        tw_start, tw_end: временное окно обслуживания, мин от начала смены.
        service: длительность обслуживания (разгрузки), мин.
    """
    id: int
    x: float
    y: float
    demand: int
    urgency: int
    tw_start: float
    tw_end: float
    service: float = 10.0


@dataclass
class Vehicle:
    """Транспортное средство (все машины парка одинаковы)."""
    id: int
    capacity: int


@dataclass
class Instance:
    """Экземпляр задачи: склад, клиенты, парк машин и параметры смены.
```

```
    Атрибуты:
        depot: координаты склада (x, y).
        clients: список клиентов; clients[i-1] имеет id = i.
        n_vehicles: число машин K.
        capacity: грузоподъёмность одной машины.
        speed: скорость движения, км/мин (постоянная).
        horizon: длительность смены, мин (склад закрывается).
        name: имя задачи (для файлов и отчётов).
    """
    depot: tuple[float, float]
    clients: list[Client]
    n_vehicles: int
    capacity: int
    speed: float = 0.5
    horizon: float = 480.0
    name: str = "instance"
    _dist: np.ndarray | None = field(default=None, repr=False, compare=False)

    @property
    def n_clients(self) -> int:
        return len(self.clients)

    @property
    def vehicles(self) -> list[Vehicle]:
        return [Vehicle(k, self.capacity) for k in range(self.n_vehicles)]

    @property
    def total_demand(self) -> int:
        return sum(c.demand for c in self.clients)

    def coords(self) -> np.ndarray:
        """Массив координат: строка 0 — склад, строка i — клиент i."""
        pts = [self.depot] + [(c.x, c.y) for c in self.clients]
        return np.asarray(pts, dtype=float)

    @property
    def dist(self) -> np.ndarray:
        """Матрица евклидовых расстояний (N+1)×(N+1), вычисляется один раз."""
        if self._dist is None:
            p = self.coords()
            diff = p[:, None, :] - p[None, :, :]
            self._dist = np.sqrt((diff ** 2).sum(axis=2))
        return self._dist

    # ---------- сериализация в JSON (для воспроизводимости экспериментов)
----------
    def to_dict(self) -> dict:
        return {
            "name": self.name,
            "depot": list(self.depot),
            "n_vehicles": self.n_vehicles,
            "capacity": self.capacity,
            "speed": self.speed,
            "horizon": self.horizon,
            "clients": [asdict(c) for c in self.clients],
        }

    @classmethod
    def from_dict(cls, d: dict) -> "Instance":
        return cls(
            depot=tuple(d["depot"]),
            clients=[Client(**c) for c in d["clients"]],
            n_vehicles=d["n_vehicles"],
            capacity=d["capacity"],
            speed=d.get("speed", 0.5),
            horizon=d.get("horizon", 480.0),
            name=d.get("name", "instance"),
        )

    def save(self, path: str | Path) -> None:
```

```
        Path(path).write_text(json.dumps(self.to_dict(), ensure_ascii=False,
indent=2),
                              encoding="utf-8")

    @classmethod
    def load(cls, path: str | Path) -> "Instance":
        return cls.from_dict(json.loads(Path(path).read_text(encoding="utf-8")))


@dataclass
class RouteStats:
    """Показатели одного маршрута (одной машины)."""
    clients: list[int]
    distance: float
    load: int
    duration: float          # время возврата на склад, мин
    lateness: float          # суммарное опоздание, мин
    n_late: int              # число клиентов, обслуженных с опозданием
    arrivals: list[float]    # время начала обслуживания каждого клиента


@dataclass
class SolutionMetrics:
    """Сводные метрики решения — «сырьё» для нечёткой оценки качества."""
    total_distance: float
    total_lateness: float
    n_late: int
    overload: float          # суммарное превышение вместимости по всем машинам
    max_load_ratio: float    # максимальная загрузка машины, доля вместимости
    lateness_index: float    # средняя относительная величина опоздания [0..1]
    urgency_index: float     # средневзвешенное нормированное время прибытия к
срочным
    balance_cv: float        # коэффициент вариации длительностей маршрутов
    n_used: int              # число задействованных машин
    routes: list[RouteStats]


# Эталонная величина опоздания для нормировки: опоздание на 60 мин и более
# считается «полным» (вклад клиента в индекс опозданий = 1).
LATENESS_REF = 60.0


@dataclass
class Solution:
    """Решение задачи: список маршрутов, каждый — последовательность id клиентов.

    Склад (0) в маршрутах не хранится: каждый маршрут начинается и
    заканчивается на складе неявно.
    """
    routes: list[list[int]]

    def evaluate(self, inst: Instance) -> SolutionMetrics:
        return evaluate_routes(self.routes, inst)

    def to_dict(self) -> dict:
        return {"routes": self.routes}


def simulate_route(route: list[int], inst: Instance) -> RouteStats:
    """Моделирует движение машины по маршруту склад → клиенты → склад.

    Если машина приезжает раньше начала окна, она ждёт; если позже конца
    окна — фиксируется опоздание (окна мягкие).
    """
    d = inst.dist
    t = 0.0
    prev = 0
    dist = 0.0
    load = 0
    late = 0.0
```

```
    n_late = 0
    arrivals = []
    for cid in route:
        c = inst.clients[cid - 1]
        leg = float(d[prev, cid])
        dist += leg
        t += leg / inst.speed
        if t < c.tw_start:            # ожидание открытия окна
            t = c.tw_start
        arrivals.append(t)
        if t > c.tw_end:              # опоздание
            late += t - c.tw_end
            n_late += 1
        t += c.service
        load += c.demand
        prev = cid
    if route:
        leg = float(d[prev, 0])
        dist += leg
        t += leg / inst.speed
    return RouteStats(list(route), float(dist), load, t if route else 0.0, late,
n_late, arrivals)


def evaluate_routes(routes: list[list[int]], inst: Instance) -> SolutionMetrics:
    """Вычисляет все метрики решения, используемые нечёткой моделью."""
    stats = [simulate_route(r, inst) for r in routes]
    cap = inst.capacity
    total_dist = sum(s.distance for s in stats)
    total_late = sum(s.lateness for s in stats)
    n_late = sum(s.n_late for s in stats)
    overload = sum(max(0, s.load - cap) for s in stats)
    max_load = max((s.load for s in stats), default=0) / cap

    # Индекс опозданий: среднее по клиентам min(1, опоздание / 60 мин).
    lat_sum = 0.0
    # Индекс срочности: средневзвешенное (вес = urgency − 1) время начала
    # обслуживания, нормированное к длительности смены.
    urg_num = 0.0
    urg_den = 0.0
    for s in stats:
        for cid, t in zip(s.clients, s.arrivals):
            c = inst.clients[cid - 1]
            if t > c.tw_end:
                lat_sum += min(1.0, (t - c.tw_end) / LATENESS_REF)
            w = c.urgency - 1
            if w > 0:
                urg_num += w * min(1.0, t / inst.horizon)
                urg_den += w
    n = max(1, inst.n_clients)
    lateness_index = lat_sum / n
    urgency_index = urg_num / urg_den if urg_den > 0 else 0.0

    # Сбалансированность: коэффициент вариации длительностей маршрутов
    # (по всем K машинам, неиспользуемая машина имеет длительность 0).
    durs = np.array([s.duration for s in stats], dtype=float)
    if len(durs) < inst.n_vehicles:
        durs = np.concatenate([durs, np.zeros(inst.n_vehicles - len(durs))])
    mean = durs.mean() if len(durs) else 0.0
    cv = float(durs.std() / mean) if mean > 0 else 0.0

    return SolutionMetrics(
        total_distance=float(total_dist),
        total_lateness=float(total_late),
        n_late=n_late,
        overload=float(overload),
        max_load_ratio=float(max_load),
        lateness_index=lateness_index,
        urgency_index=urgency_index,
        balance_cv=cv,
```

```
        n_used=sum(1 for r in routes if r),
        routes=stats,
    )


def solution_space_size(n_clients: int, n_vehicles: int) -> float:
    """Число различных хромосом (перестановка N клиентов и K−1 одинаковых
    разделителей): (N+K−1)! / (K−1)!. Возвращается как float из-за роста."""
    return math.factorial(n_clients + n_vehicles - 1) / math.factorial(n_vehicles -
1)
```

Листинг А.2 – Модуль src/fuzzy/membership.py – функции принадлежности

```
"""Функции принадлежности нечётких множеств.

Реализованы треугольная и трапециевидная функции — самые распространённые
в нечётких системах: они задаются несколькими числами, легко
интерпретируются экспертом и быстро вычисляются.
Обе функции принимают как число, так и массив numpy.
"""
from __future__ import annotations

import numpy as np


class TrapezoidalMF:
    """Трапециевидная функция принадлежности с параметрами a ≤ b ≤ c ≤ d.

            0,               x ≤ a или x ≥ d
            (x − a)/(b − a), a < x < b      (левый склон)
    μ(x) =  1,               b ≤ x ≤ c      (ядро)
            (d − x)/(d − c), c < x < d      (правый склон)

    Если a = b (или c = d), левый (правый) склон вертикальный — так задаются
    «плечевые» термы на границах универсума (например, «короткая»).
    """

    def __init__(self, a: float, b: float, c: float, d: float):
        if not a <= b <= c <= d:
            raise ValueError(f"Нарушен порядок параметров: {a}, {b}, {c}, {d}")
        self.a, self.b, self.c, self.d = float(a), float(b), float(c), float(d)

    @property
    def params(self) -> tuple[float, ...]:
        return (self.a, self.b, self.c, self.d)

    def __call__(self, x):
        a, b, c, d = self.a, self.b, self.c, self.d
        if isinstance(x, (int, float)):           # быстрый путь для одного числа
            x = float(x)
            if x < a or x > d:
                return 0.0
            if x < b:
                return (x - a) / (b - a)
            if x <= c:
                return 1.0
            return (d - x) / (d - c)
        x = np.asarray(x, dtype=float)
        # левый склон (линейный рост); при a == b склон вертикальный
        left = (x - a) / (b - a) if b > a else np.where(x >= a, 1.0, 0.0)
        # правый склон (линейный спад); при c == d склон вертикальный
        right = (d - x) / (d - c) if d > c else np.where(x <= d, 1.0, 0.0)
        # в ядре оба склона ≥ 1, вне носителя один из них < 0 — обрезаем до [0, 1]
        mu = np.clip(np.minimum(left, right), 0.0, 1.0)
        return float(mu) if mu.ndim == 0 else mu

    def __repr__(self) -> str:
        return f"Trap({self.a:g}, {self.b:g}, {self.c:g}, {self.d:g})"
```

```
class TriangularMF(TrapezoidalMF):
    """Треугольная функция принадлежности с параметрами a ≤ b ≤ c.

    Частный случай трапеции с вырожденным ядром (b = c):
    μ(x) = max(0, min((x − a)/(b − a), (c − x)/(c − b))).
    """

    def __init__(self, a: float, b: float, c: float):
        super().__init__(a, b, b, c)

    @property
    def params(self) -> tuple[float, ...]:
        return (self.a, self.b, self.d)

    def __repr__(self) -> str:
        return f"Tri({self.a:g}, {self.b:g}, {self.d:g})"
```

Листинг А.3 – Модуль src/fuzzy/variables.py – лингвистические переменные

```
"""Лингвистические переменные нечёткой модели оценки качества маршрутов.

Лингвистическая переменная — это переменная, значениями которой являются
слова (термы), например «короткая», «средняя», «длинная». Каждый терм —
нечёткое множество на универсуме (числовой области определения) с
собственной функцией принадлежности.

Пять входных переменных описывают решение задачи маршрутизации целиком,
одна выходная — его итоговое качество.
"""
from __future__ import annotations

from dataclasses import dataclass, field

from src.fuzzy.membership import TrapezoidalMF, TriangularMF


@dataclass
class Term:
    """Терм лингвистической переменной: имя, подпись и функция принадлежности."""
    name: str
    label: str
    mf: TrapezoidalMF


@dataclass
class LinguisticVariable:
    """Лингвистическая переменная.

    Атрибуты:
        name: идентификатор в программе (латиница).
        title: название для человека («Протяжённость»).
        universe: границы универсума (min, max).
        terms: термы переменной в порядке «от хорошего к плохому» или по
возрастанию.
        description: как вычисляется чёткое значение.
    """
    name: str
    title: str
    universe: tuple[float, float]
    terms: dict[str, Term] = field(default_factory=dict)
    description: str = ""

    def add(self, name: str, label: str, mf: TrapezoidalMF) -> "LinguisticVariable":
        self.terms[name] = Term(name, label, mf)
        return self
```

```
    def clip(self, x: float) -> float:
        """Приведение чёткого значения к границам универсума."""
        lo, hi = self.universe
        return min(max(x, lo), hi)

    def fuzzify(self, x: float) -> dict[str, float]:
        """Фаззификация: степени принадлежности x ко всем термам переменной."""
        x = self.clip(x)
        return {t.name: float(t.mf(x)) for t in self.terms.values()}


def make_input_variables() -> dict[str, LinguisticVariable]:
    """Создаёт пять входных лингвистических переменных.

    Параметры функций принадлежности подобраны по диапазонам значений,
    наблюдаемым на сгенерированных задачах (от случайных решений до
    решений без опозданий). Соседние термы образуют разбиение Руспини:
    склоны соседних термов пересекаются на уровне 0,5, сумма степеней
    принадлежности равна 1. Поэтому почти любое значение принадлежит
    сразу двум термам, и оценка качества меняется плавно — у ГА нет
    «плато», на которых приспособленность не реагирует на улучшения.
    """
    length = LinguisticVariable(
        "length", "Протяжённость", (0.6, 2.6),
        description="суммарный пробег, делённый на пробег жадного решения "
                    "«ближайший сосед» (1,0 — как у жадного алгоритма)")
    length.add("short", "короткая", TrapezoidalMF(0.6, 0.6, 0.8, 1.1))
    length.add("medium", "средняя", TriangularMF(0.8, 1.1, 1.6))
    length.add("long", "длинная", TrapezoidalMF(1.1, 1.6, 2.6, 2.6))

    load = LinguisticVariable(
        "load", "Загрузка транспорта", (0.0, 2.0),
        description="максимальная загрузка машины в долях от вместимости")
    load.add("low", "низкая", TrapezoidalMF(0.0, 0.0, 0.5, 0.8))
    load.add("normal", "нормальная", TrapezoidalMF(0.5, 0.8, 1.0, 1.1))
    load.add("over", "перегрузка", TrapezoidalMF(1.0, 1.1, 2.0, 2.0))

    tw = LinguisticVariable(
        "timewin", "Соблюдение временных окон", (0.0, 1.0),
        description="средняя по клиентам величина min(1, опоздание / 60 мин)")
    tw.add("good", "хорошее", TriangularMF(0.0, 0.0, 0.12))
    tw.add("fair", "удовлетворительное", TriangularMF(0.0, 0.12, 0.45))
    tw.add("poor", "плохое", TrapezoidalMF(0.12, 0.45, 1.0, 1.0))

    urg = LinguisticVariable(
        "urgency", "Обслуживание срочных заказов", (0.0, 1.0),
        description="средневзвешенное по срочности время начала обслуживания "
                    "срочных клиентов, делённое на длительность смены")
    urg.add("good", "хорошее", TrapezoidalMF(0.0, 0.0, 0.05, 0.25))
    urg.add("medium", "среднее", TriangularMF(0.05, 0.25, 0.5))
    urg.add("poor", "плохое", TrapezoidalMF(0.25, 0.5, 1.0, 1.0))

    bal = LinguisticVariable(
        "balance", "Сбалансированность", (0.0, 1.5),
        description="коэффициент вариации длительностей маршрутов машин")
    bal.add("even", "равномерная", TrapezoidalMF(0.0, 0.0, 0.05, 0.4))
    bal.add("uneven", "неравномерная", TrapezoidalMF(0.05, 0.4, 1.5, 1.5))

    return {v.name: v for v in (length, load, tw, urg, bal)}


def make_output_variable() -> LinguisticVariable:
    """Выходная переменная «Качество решения» на универсуме [0, 1]."""
    q = LinguisticVariable("quality", "Качество решения", (0.0, 1.0),
                           description="интегральная оценка плана маршрутов")
    q.add("very_low", "очень низкое", TriangularMF(0.0, 0.0, 0.25))
    q.add("low", "низкое", TriangularMF(0.0, 0.25, 0.5))
    q.add("medium", "среднее", TriangularMF(0.25, 0.5, 0.75))
    q.add("high", "высокое", TriangularMF(0.5, 0.75, 1.0))
    q.add("very_high", "очень высокое", TriangularMF(0.75, 1.0, 1.0))
```

```
    return q
```

Листинг А.4 – Модуль src/fuzzy/rules.py – база нечётких правил

```
"""База нечётких продукционных правил «ЕСЛИ – ТО» с весовыми коэффициентами.

Формат правила (по методическим указаниям, раздел 3.3.2):
    RULE_j: ЕСЛИ «условие_1» И … И «условие_m» ТО «заключение» (F_j),
где F_j ∈ (0, 1] — весовой коэффициент (степень уверенности эксперта
в истинности заключения). Каждое правило снабжено пояснением.

Правила 1–9 перебирают все сочетания термов «Протяжённости» и «Соблюдения
временных окон». Так как термы каждой переменной покрывают весь универсум,
при любых входных данных срабатывает хотя бы одно из них (полнота базы
правил), и дефаззификация всегда определена.
"""
from __future__ import annotations

from dataclasses import dataclass


@dataclass(frozen=True)
class Rule:
    """Нечёткое продукционное правило.

    Атрибуты:
        conditions: подусловия (имя_переменной, имя_терма), связанные «И».
        conclusion: имя терма выходной переменной «Качество решения».
        weight: весовой коэффициент F.
        explanation: пояснение эксперта.
    """
    conditions: tuple[tuple[str, str], ...]
    conclusion: str
    weight: float
    explanation: str


def R(cond: dict, concl: str, w: float, expl: str) -> Rule:
    return Rule(tuple(cond.items()), concl, w, expl)


RULES: list[Rule] = [
    # --- 1–9. Ядро базы: таблица «Протяжённость × Соблюдение окон» ---
    # Таблица монотонна: улучшение любого из двух показателей на один терм
    # повышает заключение на один терм (от «очень низкого» до «очень высокого»).
    R({"length": "short", "timewin": "good"}, "very_high", 1.0,
      "Короткий пробег без опозданий — главная цель планирования доставки."),
    R({"length": "short", "timewin": "fair"}, "high", 1.0,
      "Пробег экономичен, опоздания единичны и невелики."),
    R({"length": "short", "timewin": "poor"}, "medium", 1.0,
      "Экономия пробега получена ценой массовых опозданий — компромиссный план."),
    R({"length": "medium", "timewin": "good"}, "high", 1.0,
      "Опозданий нет, перерасход пробега умеренный."),
    R({"length": "medium", "timewin": "fair"}, "medium", 1.0,
      "Средний пробег и отдельные опоздания — план посредственный."),
    R({"length": "medium", "timewin": "poor"}, "low", 1.0,
      "Пробег не сокращён, а клиенты обслуживаются с опозданием."),
    R({"length": "long", "timewin": "good"}, "medium", 1.0,
      "Окна соблюдены, но за счёт большого перерасхода топлива и времени."),
    R({"length": "long", "timewin": "fair"}, "low", 1.0,
      "Длинные маршруты не спасают от опозданий — план неэффективен."),
    R({"length": "long", "timewin": "poor"}, "very_low", 1.0,
      "Большой пробег и массовые опоздания одновременно — план следует
отвергнуть."),
    # --- 10–15. Обслуживание срочных заказов ---
    # Положительные правила срабатывают только вместе с хорошими значениями
    # пробега или окон: хороший сервис срочных клиентов не «спасает» плохой план.
    R({"urgency": "good", "timewin": "good"}, "very_high", 0.8,
```

```
      "Срочные клиенты обслужены в начале смены, опозданий нет."),
    R({"urgency": "good", "length": "short"}, "high", 0.7,
      "Срочные заказы доставлены первыми без перерасхода пробега."),
    R({"urgency": "medium", "timewin": "good"}, "high", 0.6,
      "Опозданий нет, срочные заказы доставлены не первыми, но вовремя."),
    R({"urgency": "medium", "timewin": "fair"}, "medium", 0.6,
      "Посредственное обслуживание и срочных, и обычных клиентов."),
    R({"urgency": "poor"}, "low", 0.8,
      "Срочные заказы доставляются в конце смены — риск потери клиентов."),
    R({"urgency": "poor", "timewin": "poor"}, "very_low", 1.0,
      "Опоздания вместе с поздней доставкой срочных заказов — худший сервис."),
    # --- 16–18. Загрузка транспорта ---
    R({"load": "over"}, "very_low", 1.0,
      "Перегруз машины физически недопустим (дополнительно штрафуется вне нечёткого
блока)."),
    R({"load": "normal", "length": "short", "timewin": "good"}, "very_high", 0.8,
      "Рациональная загрузка кузова при коротком пробеге и без опозданий."),
    R({"load": "low", "length": "long"}, "low", 0.8,
      "Машины недогружены, но много ездят — парк используется неэффективно."),
    # --- 19–23. Сбалансированность нагрузки водителей ---
    R({"balance": "even", "timewin": "good"}, "high", 0.7,
      "Нагрузка водителей равномерна, сверхурочных нет, клиенты обслужены
вовремя."),
    R({"balance": "even", "length": "short"}, "high", 0.6,
      "Равные по длительности маршруты при экономичном пробеге."),
    R({"balance": "uneven"}, "low", 0.4,
      "Один водитель перегружен, другие простаивают."),
    R({"balance": "uneven", "timewin": "fair"}, "low", 0.7,
      "Перегруженный водитель опаздывает, пока другие простаивают."),
    R({"balance": "uneven", "length": "medium"}, "low", 0.6,
      "Неравномерная нагрузка при среднем пробеге — признак неудачного распределения
заказов."),
    # --- 24–26. Комбинированные правила ---
    R({"length": "short", "urgency": "good", "balance": "even"}, "very_high", 0.9,
      "Экономичный план с приоритетом срочных заказов и равной нагрузкой
водителей."),
    R({"length": "long", "urgency": "poor"}, "very_low", 0.8,
      "Длинные маршруты, на которых срочные заказы доставляются последними."),
    R({"length": "medium", "urgency": "good"}, "high", 0.6,
      "Небольшой перерасход пробега оправдан ранней доставкой срочных заказов."),
]


def rule_to_text(rule: Rule, variables: dict, output) -> str:
    """Текстовая запись правила: «ЕСЛИ Протяжённость = короткая И … ТО Качество =
…»."""
    conds = " И ".join(
        f"{variables[v].title} = «{variables[v].terms[t].label}»" for v, t in
rule.conditions)
    return f"ЕСЛИ {conds} ТО {output.title} =
«{output.terms[rule.conclusion].label}» " \
           f"(F = {rule.weight:g})"
```

Листинг А.5 – Модуль src/fuzzy/mamdani.py – механизм вывода Мамдани

```
"""Механизм нечёткого логического вывода по алгоритму Мамдани.

Этапы (по методическим указаниям, раздел 3.3.2):
    1. Формирование базы правил       — конструктор MamdaniInference;
    2. Фаззификация входных переменных — fuzzify:     b_i = μ(a_i);
    3. Агрегирование подусловий         — aggregate:   c_j = min{b_i};
    4. Активизация подзаключений        — activate:    d_j = c_j·F_j,
                                                       μ'_j(x) = min{d_j, μ_j(x)};
    5. Аккумуляция заключений           — accumulate:  μ_E(x) = max_j μ'_j(x);
    6. Дефаззификация (центр тяжести)   — defuzzify:   y = ∫x·μ_E(x)dx / ∫μ_E(x)dx.

Универсум выходной переменной дискретизируется сеткой из ``resolution``
точек, интегралы центра тяжести заменяются суммами (метод прямоугольников).
```

```
"""
from __future__ import annotations

from dataclasses import dataclass

import numpy as np

from src.fuzzy.rules import Rule
from src.fuzzy.variables import LinguisticVariable


@dataclass
class InferenceTrace:
    """Полный протокол одного нечёткого вывода (для отчёта и GUI)."""
    inputs: dict[str, float]                        # чёткие входные значения a_i
    fuzzified: dict[str, dict[str, float]]          # b: переменная → терм → μ
    strengths: list[float]                           # c_j — степень истинности
условий
    activations: list[float]                         # d_j = c_j · F_j
    grid: np.ndarray                                 # сетка универсума выхода
    activated_sets: np.ndarray                       # μ'_j(x), матрица n_rules ×
len(grid)
    accumulated: np.ndarray                          # μ_E(x)
    output: float                                    # y — результат дефаззификации


class MamdaniInference:
    """Нечёткая система вывода Мамдани с одной выходной переменной."""

    def __init__(self, inputs: dict[str, LinguisticVariable], output:
LinguisticVariable,
                 rules: list[Rule], resolution: int = 201):
        # Этап 1. Формирование базы правил: проверяем, что все переменные и
        # термы, упомянутые в правилах, существуют.
        for j, r in enumerate(rules, 1):
            for v, t in r.conditions:
                if v not in inputs or t not in inputs[v].terms:
                    raise ValueError(f"Правило {j}: неизвестный терм {v}={t}")
            if r.conclusion not in output.terms:
                raise ValueError(f"Правило {j}: неизвестный выходной терм
{r.conclusion}")
        self.inputs = inputs
        self.output = output
        self.rules = rules
        lo, hi = output.universe
        self.grid = np.linspace(lo, hi, resolution)
        # Функции принадлежности термов-заключений, заранее вычисленные на
        # сетке: строка j — μ терма из заключения правила j.
        self._concl_mf = np.vstack([output.terms[r.conclusion].mf(self.grid) for r
in rules])
        self._weights = np.array([r.weight for r in rules])

    # ------------------------------------------------------------------ этапы
    def fuzzify(self, values: dict[str, float]) -> dict[str, dict[str, float]]:
        """Этап 2. Фаззификация: для каждой входной переменной вычисляются
        степени принадлежности её чёткого значения ко всем термам."""
        return {name: var.fuzzify(values[name]) for name, var in
self.inputs.items()}

    def aggregate(self, fuzz: dict[str, dict[str, float]]) -> np.ndarray:
        """Этап 3. Агрегирование подусловий: степень истинности условия
        правила равна минимуму степеней истинности его подусловий (нечёткое «И»)."""
        # c_j = min{b_i} по всем подусловиям (переменная v, терм t) правила j
        return np.array([min(fuzz[v][t] for v, t in r.conditions) for r in
self.rules])

    def activate(self, c: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
        """Этап 4. Активизация: d_j = c_j·F_j; функция принадлежности
        заключения «срезается» на уровне d_j (min-активизация)."""
        d = c * self._weights                                  # d_j = c_j · F_j
```

```
        # μ'_j(x) = min(d_j, μ_j(x)): строка j — «срезанный» терм заключения правила
j
        activated = np.minimum(d[:, None], self._concl_mf)
        return d, activated

    @staticmethod
    def accumulate(activated: np.ndarray) -> np.ndarray:
        """Этап 5. Аккумуляция: объединение (max) всех активизированных множеств."""
        return activated.max(axis=0)                          # μ_E(x) = max_j
μ'_j(x)

    def defuzzify(self, mu: np.ndarray) -> float:
        """Этап 6. Дефаззификация методом центра тяжести.

        Если ни одно правило не сработало (площадь равна нулю), возвращается
        0 — худшая оценка. При полной базе правил такого не происходит.
        """
        area = mu.sum()                                       # Σ μ_E(x_g)
        if area <= 1e-12:
            return 0.0
        return float((self.grid * mu).sum() / area)           # Σ x_g·μ_E(x_g) / Σ
μ_E(x_g)

    # ------------------------------------------------------------- вывод целиком
    def infer(self, values: dict[str, float]) -> float:
        """Чёткие входы → чёткая оценка качества (этапы 2–6)."""
        fuzz = self.fuzzify(values)
        c = self.aggregate(fuzz)
        _, activated = self.activate(c)
        return self.defuzzify(self.accumulate(activated))

    def trace(self, values: dict[str, float]) -> InferenceTrace:
        """То же, что infer, но с сохранением всех промежуточных результатов."""
        fuzz = self.fuzzify(values)
        c = self.aggregate(fuzz)
        d, activated = self.activate(c)
        acc = self.accumulate(activated)
        return InferenceTrace(dict(values), fuzz, c.tolist(), d.tolist(), self.grid,
                              activated, acc, self.defuzzify(acc))


def build_default_engine(resolution: int = 201) -> MamdaniInference:
    """Создаёт систему вывода с переменными и базой правил по умолчанию."""
    from src.fuzzy.rules import RULES
    from src.fuzzy.variables import make_input_variables, make_output_variable
    return MamdaniInference(make_input_variables(), make_output_variable(), RULES,
resolution)
```

Листинг А.6 – Модуль src/ga/encoding.py – кодирование хромосомы

```
"""Кодирование решения в хромосому и обратно.

Хромосома — перестановка длины N+K−1 из чисел 1..N+K−1:
    * гены 1..N — номера клиентов (порядок объезда);
    * гены N+1..N+K−1 — разделители между маршрутами машин.

Пример (N=6, K=3, разделители 7 и 8):
    [3, 1, 7, 6, 2, 8, 5, 4]  →  машина 1: 3→1, машина 2: 6→2, машина 3: 5→4

Так одна хромосома одновременно задаёт и распределение заказов по машинам,
и порядок объезда внутри каждого маршрута.
"""
from __future__ import annotations

import numpy as np


def is_separator(gene: int, n_clients: int) -> bool:
```

```
    return gene > n_clients


def decode(chrom, n_clients: int) -> list[list[int]]:
    """Хромосома → список маршрутов (разрезание по разделителям)."""
    routes: list[list[int]] = [[]]
    for g in chrom:
        if g > n_clients:
            routes.append([])
        else:
            routes[-1].append(int(g))
    return routes


def encode(routes: list[list[int]], n_clients: int) -> list[int]:
    """Список маршрутов → хромосома (маршруты, склеенные через разделители)."""
    chrom: list[int] = []
    sep = n_clients + 1
    for i, r in enumerate(routes):
        if i > 0:
            chrom.append(sep)
            sep += 1
        chrom.extend(r)
    return chrom


def random_chromosome(n_clients: int, n_vehicles: int, rng: np.random.Generator) ->
list[int]:
    """Случайная хромосома: случайная перестановка клиентов и разделителей."""
    genes = np.arange(1, n_clients + n_vehicles)
    rng.shuffle(genes)
    return genes.tolist()


def is_valid(chrom, n_clients: int, n_vehicles: int) -> bool:
    """Проверка, что хромосома — перестановка 1..N+K−1 (каждый ген ровно один
раз)."""
    return sorted(chrom) == list(range(1, n_clients + n_vehicles))
```

Листинг А.7 – Модуль src/ga/operators.py – генетические операторы

```
"""Генетические операторы: отбор, скрещивание, мутации.

Все операторы работают с хромосомой-перестановкой (см. encoding.py) и
сохраняют её корректность: каждый клиент и каждый разделитель после
операции встречается ровно один раз.
"""
from __future__ import annotations

import numpy as np


# ============================================================================ ОТБОР
def tournament_selection(fits: np.ndarray, rng: np.random.Generator, k: int = 3) ->
int:
    """Турнирный отбор: из k случайных особей выбирается лучшая.

    Давление отбора регулируется размером турнира k; не зависит от масштаба
    значений приспособленности (важно, т.к. штраф делает их отрицательными).
    """
    idx = rng.integers(0, len(fits), size=k)
    return int(idx[np.argmax(fits[idx])])


def roulette_selection(fits: np.ndarray, rng: np.random.Generator, **_) -> int:
    """Пропорциональный (рулеточный) отбор.

    Вероятность выбора особи пропорциональна её приспособленности,
```

```
    предварительно сдвинутой так, чтобы минимум стал малым положительным
    числом (приспособленность со штрафом может быть отрицательной).
    """
    shifted = fits - fits.min() + 1e-6
    p = shifted / shifted.sum()
    return int(rng.choice(len(fits), p=p))


def rank_selection(fits: np.ndarray, rng: np.random.Generator, **_) -> int:
    """Ранговый отбор: вероятность пропорциональна рангу особи (1 — худшая,
    P — лучшая). Устраняет доминирование «сверхособей» на ранних поколениях."""
    ranks = np.empty(len(fits))
    ranks[np.argsort(fits)] = np.arange(1, len(fits) + 1)
    return int(rng.choice(len(fits), p=ranks / ranks.sum()))


SELECTION = {
    "tournament": tournament_selection,
    "roulette": roulette_selection,
    "rank": rank_selection,
}


# ================================================================== СКРЕЩИВАНИЕ
def order_crossover(p1: list[int], p2: list[int], rng: np.random.Generator) ->
list[int]:
    """Упорядоченное скрещивание (Order Crossover, OX).

    1. Случайно выбирается отрезок [i, j) и копируется из первого родителя
       в потомка на те же позиции.
    2. Оставшиеся позиции, начиная с j и по кругу, заполняются генами второго
       родителя в порядке их следования (также начиная с позиции j),
       пропуская уже скопированные гены.
    Потомок наследует от первого родителя фрагмент маршрута, а от второго —
    относительный порядок остальных клиентов. Результат — всегда перестановка.
    """
    n = len(p1)
    i, j = sorted(rng.choice(n + 1, size=2, replace=False))
    child = [None] * n
    child[i:j] = p1[i:j]
    taken = set(p1[i:j])
    fill = [p2[(j + k) % n] for k in range(n) if p2[(j + k) % n] not in taken]
    pos = j % n
    for g in fill:
        while child[pos] is not None:
            pos = (pos + 1) % n
        child[pos] = g
        pos = (pos + 1) % n
    return child


# ================================================================== МУТАЦИИ
def swap_mutation(chrom: list[int], rng: np.random.Generator, n_clients: int) ->
list[int]:
    """Обмен двух случайных генов (в т.ч. клиентов разных маршрутов)."""
    c = list(chrom)
    i, j = rng.choice(len(c), size=2, replace=False)
    c[i], c[j] = c[j], c[i]
    return c


def inversion_mutation(chrom: list[int], rng: np.random.Generator, n_clients: int) -
> list[int]:
    """Инверсия случайного отрезка: порядок генов на нём меняется на обратный.

    Если отрезок лежит внутри одного маршрута, это аналог хода 2-opt —
    устраняет самопересечения маршрута.
    """
    c = list(chrom)
    i, j = sorted(rng.choice(len(c) + 1, size=2, replace=False))
```

```
    c[i:j] = c[i:j][::-1]
    return c


def transfer_mutation(chrom: list[int], rng: np.random.Generator, n_clients: int) ->
list[int]:
    """Перенос клиента в другой маршрут: случайный клиент удаляется из своего
    маршрута и вставляется в случайную позицию другого маршрута.

    Меняет распределение заказов по машинам, не затрагивая порядок
    остальных клиентов (важно для выравнивания загрузки).
    """
    c = list(chrom)
    client_pos = [p for p, g in enumerate(c) if g <= n_clients]
    p = int(rng.choice(client_pos))
    gene = c.pop(p)
    # номера маршрутов: маршрут r — между (r−1)-м и r-м разделителем
    seps = [q for q, g in enumerate(c) if g > n_clients]
    bounds = [-1] + seps + [len(c)]
    own = sum(1 for q in seps if q < p)         # маршрут, из которого удалён клиент
    others = [r for r in range(len(bounds) - 1) if r != own]
    if not others:                               # единственная машина
        c.insert(p, gene)
        return c
    r = int(rng.choice(others))
    lo, hi = bounds[r] + 1, bounds[r + 1]        # допустимые позиции вставки [lo,
hi]
    c.insert(int(rng.integers(lo, hi + 1)), gene)
    return c


MUTATIONS = (swap_mutation, inversion_mutation, transfer_mutation)


def mutate(chrom: list[int], rng: np.random.Generator, n_clients: int) -> list[int]:
    """Применяет один из трёх операторов мутации, выбранный равновероятно."""
    op = MUTATIONS[int(rng.integers(len(MUTATIONS)))]
    return op(chrom, rng, n_clients)
```

Листинг А.8 – Модуль src/ga/fitness.py – функции приспособленности

```
"""Функции приспособленности для генетического алгоритма.

Главная — ``FuzzyFitness``: здесь и происходит гибридизация методов ИИ.
Для каждой особи каждого поколения ГА вызывает нечёткий вывод Мамдани,
и полученная оценка качества Q ∈ [0, 1] становится приспособленностью.

Для сравнения реализована ``WeightedSumFitness`` — та же схема, но вместо
нечёткого блока используется линейная взвешенная сумма тех же показателей.
"""
from __future__ import annotations

from src.baselines import nearest_neighbor
from src.fuzzy.mamdani import MamdaniInference, build_default_engine
from src.model import Instance, SolutionMetrics, evaluate_routes

# Штраф за перегруз: любое решение с перегрузом получает приспособленность
# не выше Q_max − 1 < 0, т.е. хуже любого допустимого решения (Q ≥ 0);
# слагаемое, пропорциональное величине перегруза, сохраняет «градиент»
# среди недопустимых решений, чтобы ГА мог выбираться из этой области.
OVERLOAD_BASE = 1.0
OVERLOAD_RATE = 1.0


def overload_penalty(m: SolutionMetrics, capacity: int) -> float:
    """Штраф за нарушение жёсткого ограничения вместимости."""
    if m.overload <= 0:
        return 0.0
```

```
    return OVERLOAD_BASE + OVERLOAD_RATE * m.overload / capacity


def fuzzy_inputs(m: SolutionMetrics, ref_distance: float) -> dict[str, float]:
    """Чёткие значения пяти входных лингвистических переменных."""
    return {
        "length": m.total_distance / ref_distance,
        "load": m.max_load_ratio,
        "timewin": m.lateness_index,
        "urgency": m.urgency_index,
        "balance": m.balance_cv,
    }


class FuzzyFitness:
    """Приспособленность = нечёткая оценка качества − штраф за перегруз.

    Эталонный пробег для нормировки «Протяжённости» — пробег жадного
    решения «ближайший сосед» на той же задаче (вычисляется один раз).
    Значения кэшируются: одинаковые особи (частые после элитизма и
    скрещивания похожих родителей) повторно не оцениваются.
    """

    name = "fuzzy"

    def __init__(self, inst: Instance, engine: MamdaniInference | None = None):
        self.inst = inst
        self.engine = engine or build_default_engine()
        self.ref_distance = evaluate_routes(nearest_neighbor(inst),
inst).total_distance
        self.cache: dict[tuple, float] = {}
        self.evaluations = 0

    def quality(self, m: SolutionMetrics) -> float:
        """Нечёткая оценка Q (без штрафа) — вызов вывода Мамдани."""
        return self.engine.infer(fuzzy_inputs(m, self.ref_distance))

    def score(self, routes: list[list[int]]) -> float:
        """Приспособленность решения, заданного списком маршрутов."""
        m = evaluate_routes(routes, self.inst)
        return self.quality(m) - overload_penalty(m, self.inst.capacity)

    def __call__(self, chrom) -> float:
        from src.ga.encoding import decode
        key = tuple(chrom)
        val = self.cache.get(key)
        if val is None:                    # особь ещё не оценивалась
            self.evaluations += 1
            # хромосома → маршруты → метрики → нечёткий вывод Мамдани − штраф
            val = self.score(decode(chrom, self.inst.n_clients))
            self.cache[key] = val
        return val


class WeightedSumFitness(FuzzyFitness):
    """Альтернатива для сравнения: линейная свёртка тех же показателей.

    Q_ws = 1 − (w_L·s_L + w_T·s_T + w_U·s_U + w_B·s_B), где s — показатели,
    нормированные к [0, 1] по границам универсумов нечётких переменных.
    Перегруз штрафуется так же, как и в нечёткой схеме.
    """

    name = "weighted"
    WEIGHTS = {"length": 0.35, "timewin": 0.30, "urgency": 0.20, "balance": 0.15}

    def quality(self, m: SolutionMetrics) -> float:
        x = fuzzy_inputs(m, self.ref_distance)
        s = {}
        for k in self.WEIGHTS:
            lo, hi = self.engine.inputs[k].universe
```

```
            s[k] = (min(max(x[k], lo), hi) - lo) / (hi - lo)
        return 1.0 - sum(w * s[k] for k, w in self.WEIGHTS.items())
```

Листинг А.9 – Модуль src/ga/engine.py – цикл генетического алгоритма

```
"""Главный цикл генетического алгоритма.

Схема одного поколения:
    1. оценка приспособленности всех особей (нечёткий вывод Мамдани);
    2. элитизм — лучшие ``elite`` особей переходят в новое поколение без изменений;
    3. пока новое поколение не заполнено: отбор двух родителей →
       скрещивание OX с вероятностью pc → мутация каждого потомка с вероятностью pm;
    4. проверка критериев останова: лимит поколений, стагнация, лимит времени.
"""
from __future__ import annotations

import time
from dataclasses import dataclass, field
from typing import Callable

import numpy as np

from src.baselines import randomized_nearest_neighbor
from src.ga.encoding import decode, encode, random_chromosome
from src.ga.operators import SELECTION, mutate, order_crossover
from src.model import Instance


@dataclass
class GAConfig:
    """Параметры генетического алгоритма (значения по умолчанию обоснованы
    экспериментами раздела 4 пояснительной записки)."""
    pop_size: int = 100
    generations: int = 300
    crossover_rate: float = 0.9
    mutation_rate: float = 0.2
    selection: str = "tournament"
    tournament_k: int = 3
    elite: int = 2
    seed_fraction: float = 0.1     # доля начальной популяции из рандомизированной
эвристики
    stagnation: int = 120          # останов, если лучшая особь не улучшалась
столько поколений
    time_limit: float | None = None
    seed: int = 0


@dataclass
class GAResult:
    """Результат работы ГА и журнал сходимости."""
    best_chrom: list[int]
    best_routes: list[list[int]]
    best_fitness: float
    history_best: list[float] = field(default_factory=list)
    history_mean: list[float] = field(default_factory=list)
    history_worst: list[float] = field(default_factory=list)
    initial_best_routes: list[list[int]] | None = None
    generations_run: int = 0
    evaluations: int = 0
    elapsed: float = 0.0
    stop_reason: str = ""


def run_ga(inst: Instance, fitness: Callable[[list[int]], float], cfg: GAConfig |
None = None,
           callback: Callable[[int, float, float, list[int]], bool | None] | None =
None
           ) -> GAResult:
```

```
    """Запускает ГА на задаче ``inst`` с функцией приспособленности ``fitness``.

    callback(поколение, лучшая, средняя, лучшая_хромосома) вызывается после
    каждого поколения (используется GUI); если он вернёт True — останов.
    """
    cfg = cfg or GAConfig()
    rng = np.random.default_rng(cfg.seed)
    n, k = inst.n_clients, inst.n_vehicles
    select = SELECTION[cfg.selection]
    t0 = time.perf_counter()

    # Начальная популяция: небольшая доля (seed_fraction) — решения
    # рандомизированного «ближайшего соседа», остальные — случайные
    # перестановки. «Засев» выводит часть особей в область, где нечёткая
    # оценка информативна, а случайные особи обеспечивают разнообразие.
    n_seed = int(round(cfg.seed_fraction * cfg.pop_size))
    pop = [encode(randomized_nearest_neighbor(inst, rng), n) for _ in range(n_seed)]
    pop += [random_chromosome(n, k, rng) for _ in range(cfg.pop_size - n_seed)]
    fits = np.array([fitness(c) for c in pop])

    best_i = int(np.argmax(fits))
    best_chrom, best_fit = list(pop[best_i]), float(fits[best_i])
    res = GAResult(best_chrom, decode(best_chrom, n), best_fit,
                   initial_best_routes=decode(best_chrom, n))
    last_improve = 0
    gen = 0
    res.stop_reason = "лимит поколений"
    for gen in range(1, cfg.generations + 1):
        # --- журнал сходимости (по текущему поколению)
        res.history_best.append(float(fits.max()))
        res.history_mean.append(float(fits.mean()))
        res.history_worst.append(float(fits.min()))

        # --- элитизм: лучшие особи копируются без изменений
        order = np.argsort(fits)[::-1]
        new_pop = [list(pop[i]) for i in order[:cfg.elite]]

        # --- отбор, скрещивание, мутация
        while len(new_pop) < cfg.pop_size:
            p1 = pop[select(fits, rng, k=cfg.tournament_k)]
            p2 = pop[select(fits, rng, k=cfg.tournament_k)]
            if rng.random() < cfg.crossover_rate:
                c1, c2 = order_crossover(p1, p2, rng), order_crossover(p2, p1, rng)
            else:
                c1, c2 = list(p1), list(p2)
            for child in (c1, c2):
                if rng.random() < cfg.mutation_rate:
                    child = mutate(child, rng, n)
                if len(new_pop) < cfg.pop_size:
                    new_pop.append(child)
        pop = new_pop
        fits = np.array([fitness(c) for c in pop])

        # --- обновление лучшего решения
        i = int(np.argmax(fits))
        if fits[i] > best_fit + 1e-12:
            best_fit, best_chrom = float(fits[i]), list(pop[i])
            last_improve = gen
        if callback is not None and callback(gen, best_fit, float(fits.mean()),
best_chrom):
            res.stop_reason = "остановлено пользователем"
            break
        # --- критерии останова
        if gen - last_improve >= cfg.stagnation:
            res.stop_reason = f"стагнация {cfg.stagnation} поколений"
            break
        if cfg.time_limit is not None and time.perf_counter() - t0 > cfg.time_limit:
            res.stop_reason = "лимит времени"
            break
```

```
    res.history_best.append(float(fits.max()))
    res.history_mean.append(float(fits.mean()))
    res.history_worst.append(float(fits.min()))
    res.best_chrom, res.best_fitness = best_chrom, best_fit
    res.best_routes = decode(best_chrom, n)
    res.generations_run = gen
    res.evaluations = getattr(fitness, "evaluations", 0)
    res.elapsed = time.perf_counter() - t0
    return res
```

ПРИЛОЖЕНИЕ Б

(справочное)

Пример входного файла задачи

Задача с N = 10 клиентами и K = 2 машинами (файл experiments/instances/N10_s1.json). Поля клиента: id – номер, x и y – координаты, км; demand – объём заказа; urgency – срочность (1 – обычный, 2 – повышенный, 3 – срочный); tw_start и tw_end – временное окно, мин от начала смены; service – время разгрузки, мин.

```
{
  "name": "N10_s1", "depot": [20.0, 20.0], "n_vehicles": 2,
  "capacity": 32, "speed": 0.5, "horizon": 480.0,
  "clients": [
    {"id": 1, "x": 20.47, "y": 38.02, "demand": 1, "urgency": 2, "tw_start": 49.2,
"tw_end": 185.8, "service": 10.0},
    {"id": 2, "x": 5.77, "y": 37.95, "demand": 8, "urgency": 1, "tw_start": 96.1,
"tw_end": 261.2, "service": 10.0},
    {"id": 3, "x": 12.47, "y": 16.93, "demand": 1, "urgency": 1, "tw_start": 0.0,
"tw_end": 97.2, "service": 10.0},
    {"id": 4, "x": 33.11, "y": 16.37, "demand": 3, "urgency": 1, "tw_start": 157.6,
"tw_end": 359.9, "service": 10.0},
    {"id": 5, "x": 21.98, "y": 1.1, "demand": 5, "urgency": 3, "tw_start": 0.0,
"tw_end": 60.8, "service": 10.0},
    {"id": 6, "x": 30.14, "y": 21.53, "demand": 5, "urgency": 1, "tw_start": 136.8,
"tw_end": 333.0, "service": 10.0},
    {"id": 7, "x": 13.19, "y": 31.54, "demand": 2, "urgency": 1, "tw_start": 28.8,
"tw_end": 209.9, "service": 10.0},
    {"id": 8, "x": 12.13, "y": 18.14, "demand": 10, "urgency": 2, "tw_start": 0.0,
"tw_end": 69.0, "service": 10.0},
    {"id": 9, "x": 5.36, "y": 16.12, "demand": 8, "urgency": 2, "tw_start": 0.0,
"tw_end": 104.5, "service": 10.0},
    {"id": 10, "x": 8.14, "y": 10.49, "demand": 10, "urgency": 2, "tw_start": 10.0,
"tw_end": 133.6, "service": 10.0}
  ]
}
```

ПРИЛОЖЕНИЕ В

(справочное)

Пример протокола работы программы

Вывод консольного интерфейса при решении задачи с N = 20 клиентами (команда python -m src.main --clients 20 --seed 2 --trace): ход эволюции, сравнение с жадным алгоритмом, маршруты машин и полный протокол нечёткого вывода для найденного решения.

```
Задача N20_K3_s2: N = 20, K = 3, вместимость 38, суммарный спрос 95
  поколение   50: лучшая 0.6375, средняя 0.2926
  поколение  100: лучшая 0.6986, средняя 0.3948
  поколение  150: лучшая 0.7492, средняя 0.5787
  поколение  200: лучшая 0.7492, средняя 0.5813
ГА завершён: 247 поколений, 5815 оценок, 1.0 с, причина останова — стагнация 120
поколений
Жадный (ближ. сосед)   пробег    228.1 км | опозданий  10 (  331.5 мин) | перегруз
0 | Q
        = 0.481
ГА (fuzzy)             пробег    222.5 км | опозданий   1 (    2.5 мин) | перегруз
0 | Q
        = 0.749
  Машина 1: склад → 11 → 7 → 14 → 12 → 9 → 20 → 18 → склад
  Машина 2: склад → 6 → 10 → 1 → 15 → 4 → 2 → склад
  Машина 3: склад → 8 → 3 → 17 → 5 → 16 → 13 → 19 → склад

Протокол нечёткого вывода:
  Протяжённость                  =  0.975  →  короткая 0.42, средняя 0.58
  Загрузка транспорта            =  1.000  →  нормальная 1.00
  Соблюдение временных окон      =  0.002  →  хорошее 0.98, удовлетворительное 0.02
  Обслуживание срочных заказов   =  0.164  →  хорошее 0.43, среднее 0.57
  Сбалансированность             =  0.062  →  равномерная 0.97, неравномерная 0.03
  П1  c = 0.415, d = 0.415: ЕСЛИ Протяжённость = «короткая» И Соблюдение временных
окон =
        «хорошее» ТО Качество решения = «очень высокое» (F = 1)
  П2  c = 0.018, d = 0.018: ЕСЛИ Протяжённость = «короткая» И Соблюдение временных
окон =
        «удовлетворительное» ТО Качество решения = «высокое» (F = 1)
  П4  c = 0.585, d = 0.585: ЕСЛИ Протяжённость = «средняя» И Соблюдение временных
окон =
        «хорошее» ТО Качество решения = «высокое» (F = 1)
  П5  c = 0.018, d = 0.018: ЕСЛИ Протяжённость = «средняя» И Соблюдение временных
окон =
        «удовлетворительное» ТО Качество решения = «среднее» (F = 1)
  П10 c = 0.431, d = 0.345: ЕСЛИ Обслуживание срочных заказов = «хорошее» И
Соблюдение
        временных окон = «хорошее» ТО Качество решения = «очень высокое» (F = 0.8)
  П11 c = 0.415, d = 0.291: ЕСЛИ Обслуживание срочных заказов = «хорошее» И
Протяжённость =
        «короткая» ТО Качество решения = «высокое» (F = 0.7)
  П12 c = 0.569, d = 0.342: ЕСЛИ Обслуживание срочных заказов = «среднее» И
Соблюдение
        временных окон = «хорошее» ТО Качество решения = «высокое» (F = 0.6)
  П13 c = 0.018, d = 0.011: ЕСЛИ Обслуживание срочных заказов = «среднее» И
Соблюдение
        временных окон = «удовлетворительное» ТО Качество решения = «среднее» (F =
0.6)
  П17 c = 0.415, d = 0.332: ЕСЛИ Загрузка транспорта = «нормальная» И Протяжённость
=
        «короткая» И Соблюдение временных окон = «хорошее» ТО Качество решения =
«очень
```

```
        высокое» (F = 0.8)
  П19 c = 0.966, d = 0.676: ЕСЛИ Сбалансированность = «равномерная» И Соблюдение
временных
        окон = «хорошее» ТО Качество решения = «высокое» (F = 0.7)
  П20 c = 0.415, d = 0.249: ЕСЛИ Сбалансированность = «равномерная» И Протяжённость
=
        «короткая» ТО Качество решения = «высокое» (F = 0.6)
  П21 c = 0.034, d = 0.014: ЕСЛИ Сбалансированность = «неравномерная» ТО Качество
решения =
        «низкое» (F = 0.4)
  П22 c = 0.018, d = 0.012: ЕСЛИ Сбалансированность = «неравномерная» И Соблюдение
        временных окон = «удовлетворительное» ТО Качество решения = «низкое» (F =
0.7)
  П23 c = 0.034, d = 0.021: ЕСЛИ Сбалансированность = «неравномерная» И
Протяжённость =
        «средняя» ТО Качество решения = «низкое» (F = 0.6)
  П24 c = 0.415, d = 0.374: ЕСЛИ Протяжённость = «короткая» И Обслуживание срочных
заказов
        = «хорошее» И Сбалансированность = «равномерная» ТО Качество решения =
«очень
        высокое» (F = 0.9)
  П26 c = 0.431, d = 0.258: ЕСЛИ Протяжённость = «средняя» И Обслуживание срочных
заказов =
        «хорошее» ТО Качество решения = «высокое» (F = 0.6)
  Дефаззификация (центр тяжести): Q = 0.7492
```

ПРИЛОЖЕНИЕ Г

(справочное)

Диаграмма классов интеллектуальной системы

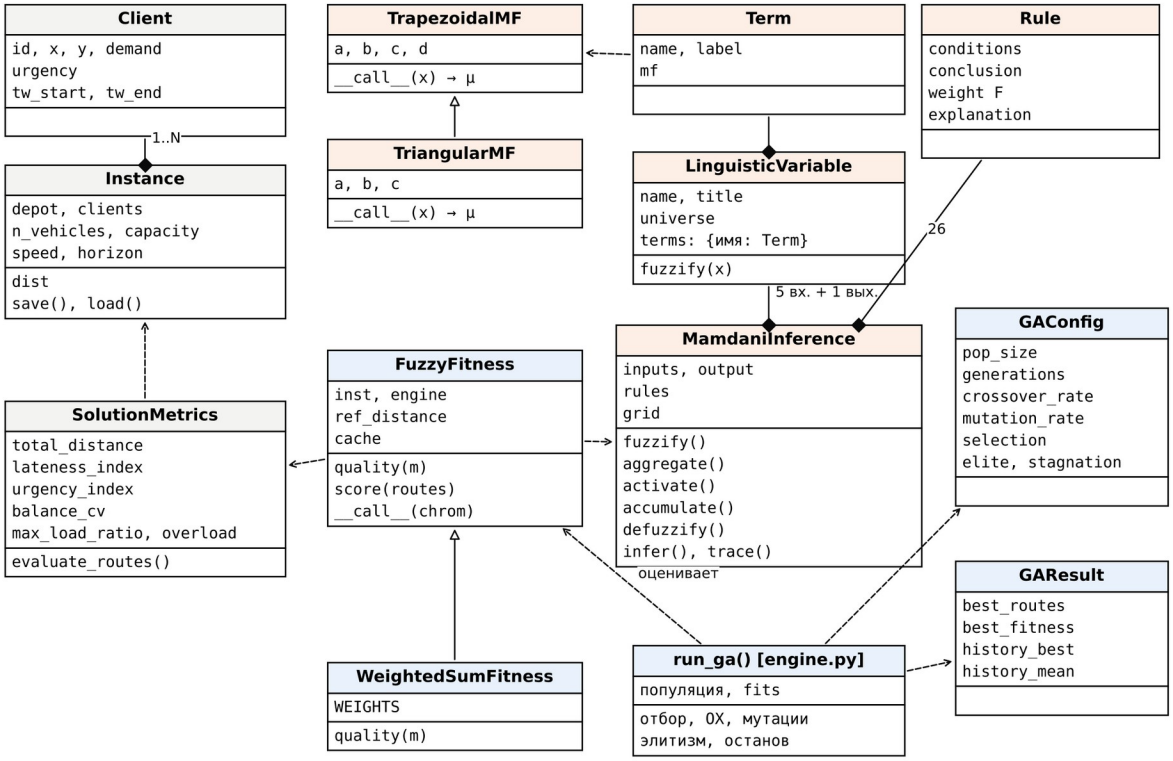


Рисунок Г.1 – Диаграмма основных классов системы